Universidade Federal da Fronteira Sul
Curso de Ciência da Computação

Redes de Computadores

Prof. Marco Aurélio Spohn

marco.spohn@uffs.edu.br

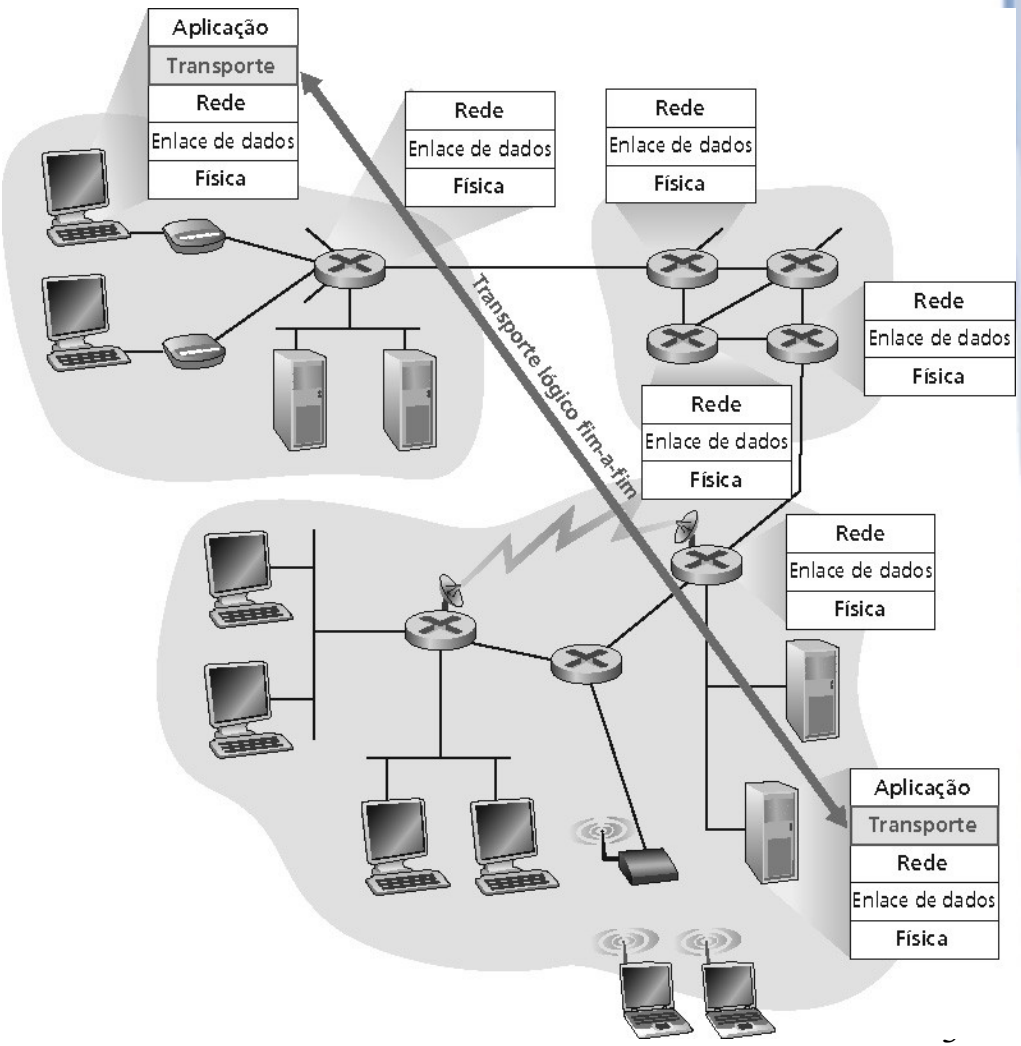
Esta apresentação contém slides fornecidos pela Editora Pearson como material de apoio ao Professor do livro "Redes de Computadores e a Internet: uma abordagem top-down", bem como material da disciplina "Computer Networks" ministrada pelo Prof. J. J. Garcia-Luna-Aceves da University of California at Santa Cruz

Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Protocolos e serviços de transporte

- Fornecem comunicação lógica entre processos de aplicação em diferentes hospedeiros
- Os protocolos de transporte são executados nos sistemas finais
  - Lado emissor: quebra as mensagens da aplicação em segmentos e envia para a camada de rede
  - Lado receptor: remonta os segmentos em mensagens e passa para a camada de aplicação
- Há mais de um protocolo de transporte disponível para as aplicações
  - Internet: TCP e UDP

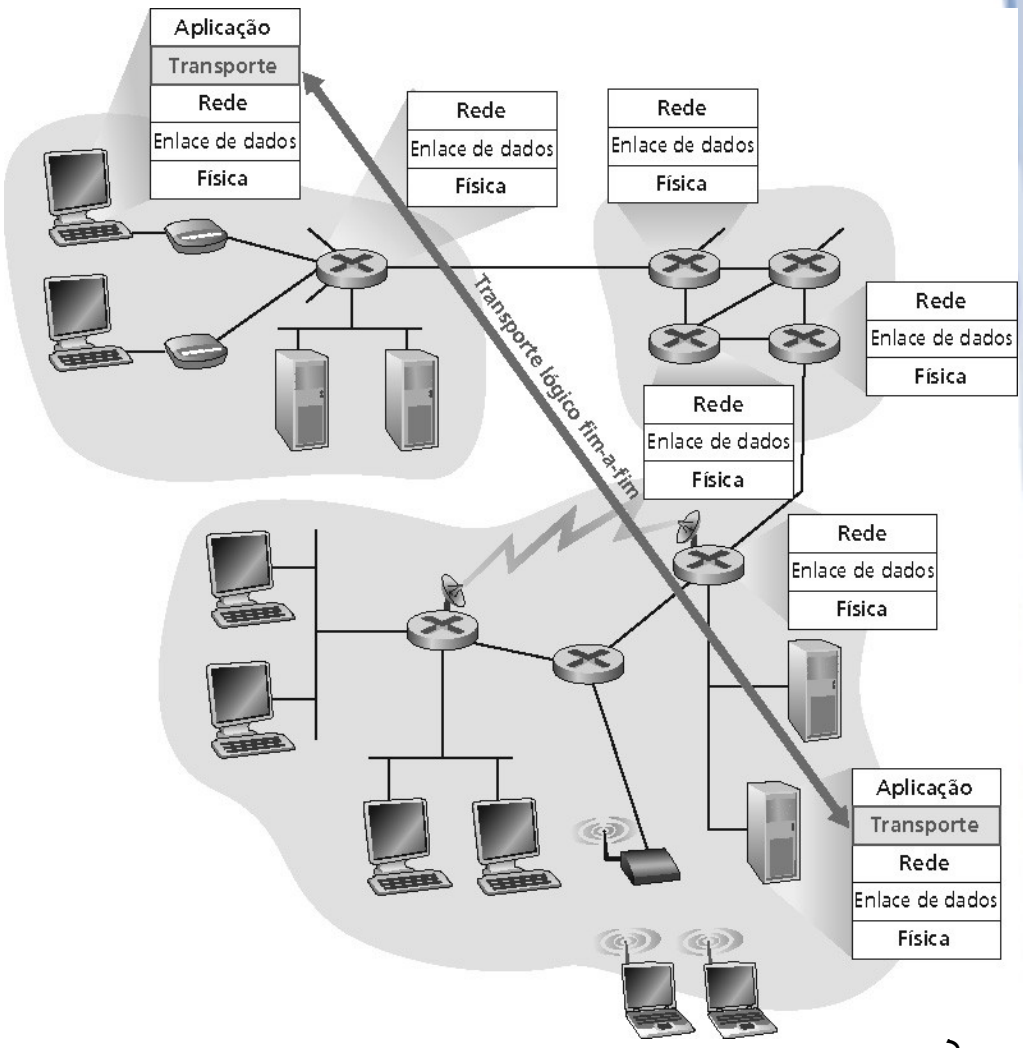


Camada de transporte *vs.* camada de rede

- Camada de rede: comunicação lógica entre os hospedeiros
- Camada de transporte: comunicação lógica entre os processos
  - Depende dos serviços da camada de rede!

Protocolos da camada de transporte da Internet

- TCP:
  - Confiável garante ordem de entrega (TCP)
  - Controle de congestionamento
  - Controle de fluxo
  - Orientado à conexão
- UDP:
  - Não confiável, não garante ordem na entrega
  - Extensão do “melhor esforço” do IP
- Serviços não disponíveis:
  - Garantia a atrasos
  - Garantia de banda
  - Por quê?



Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

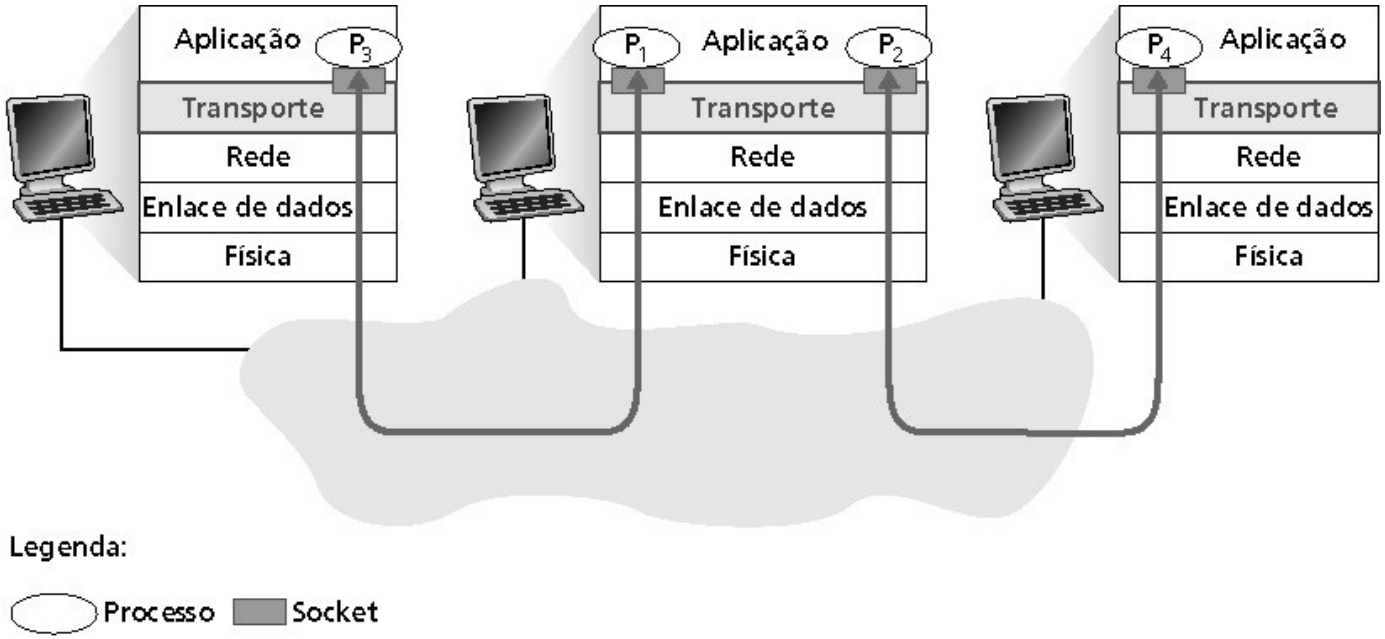
Multiplexação/demultiplexação

Demultiplexação no hospedeiro receptor:

entrega os segmentos recebidos ao socket correto

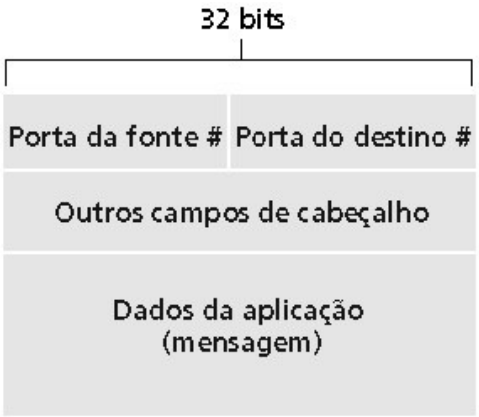
Multiplexação no hospedeiro emissor:

coleta dados de múltiplos sockets, envelopa os dados com cabeçalho (usado depois para demultiplexação)



Como funciona a demultiplexação

- Computador recebe datagramas IP
  - Cada datagrama possui endereço IP de origem e IP de destino
  - Cada datagrama carrega um segmento da camada de transporte
  - Cada segmento possui números de porta de origem e destino (lembre-se: números de porta bem conhecidos para aplicações específicas)
- O hospedeiro usa endereços IP e números de porta para direcionar o segmento ao socket apropriado



Demultiplexação não orientada à conexão

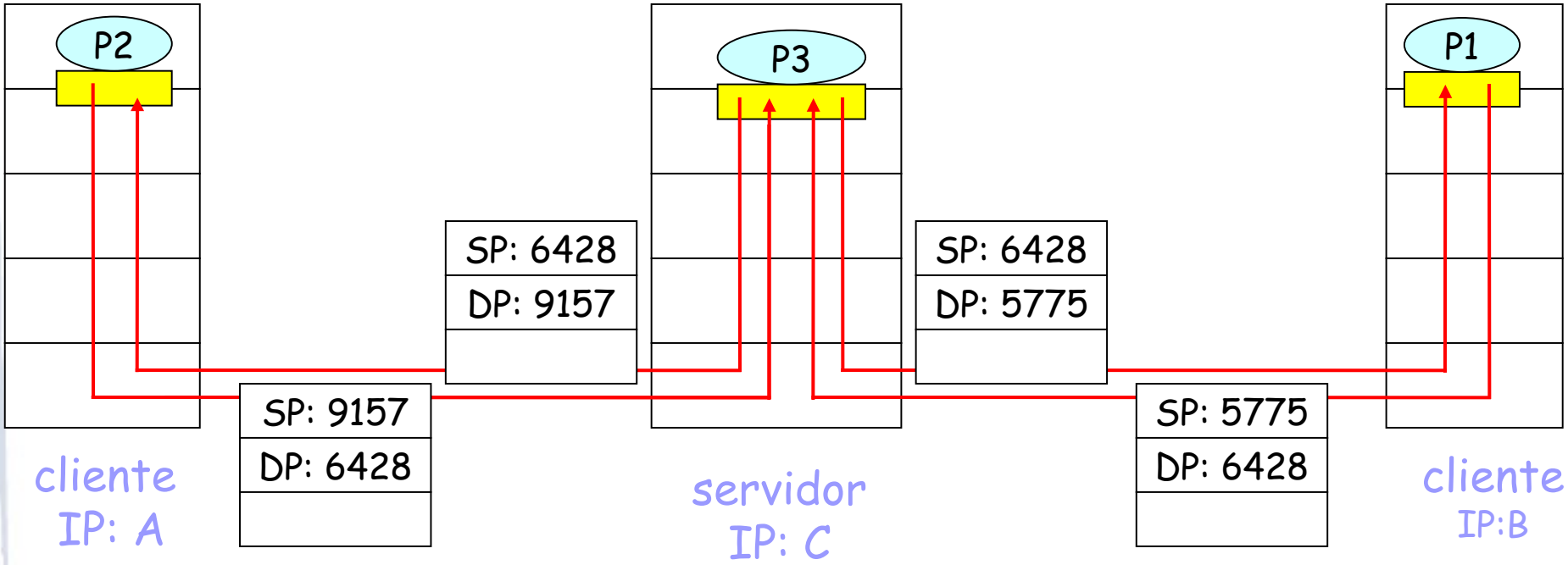
- Cria sockets com números de porta:

```
DatagramSocket mySocket1 = new DatagramSocket(9111);
DatagramSocket mySocket2 = new DatagramSocket(9222);
```

- Socket UDP identificado por 2 valores:
  (endereço IP de destino, número da porta de destino)
- Quando o hospedeiro recebe o segmento UDP:
  - Verifica o número da porta de destino no segmento
  - Direciona o segmento UDP para o socket com este número de porta
- Datagramas com IP de origem diferentes e/ou portas de origem diferentes são direcionados para o mesmo socket

Demultiplexação não orientada à conexão

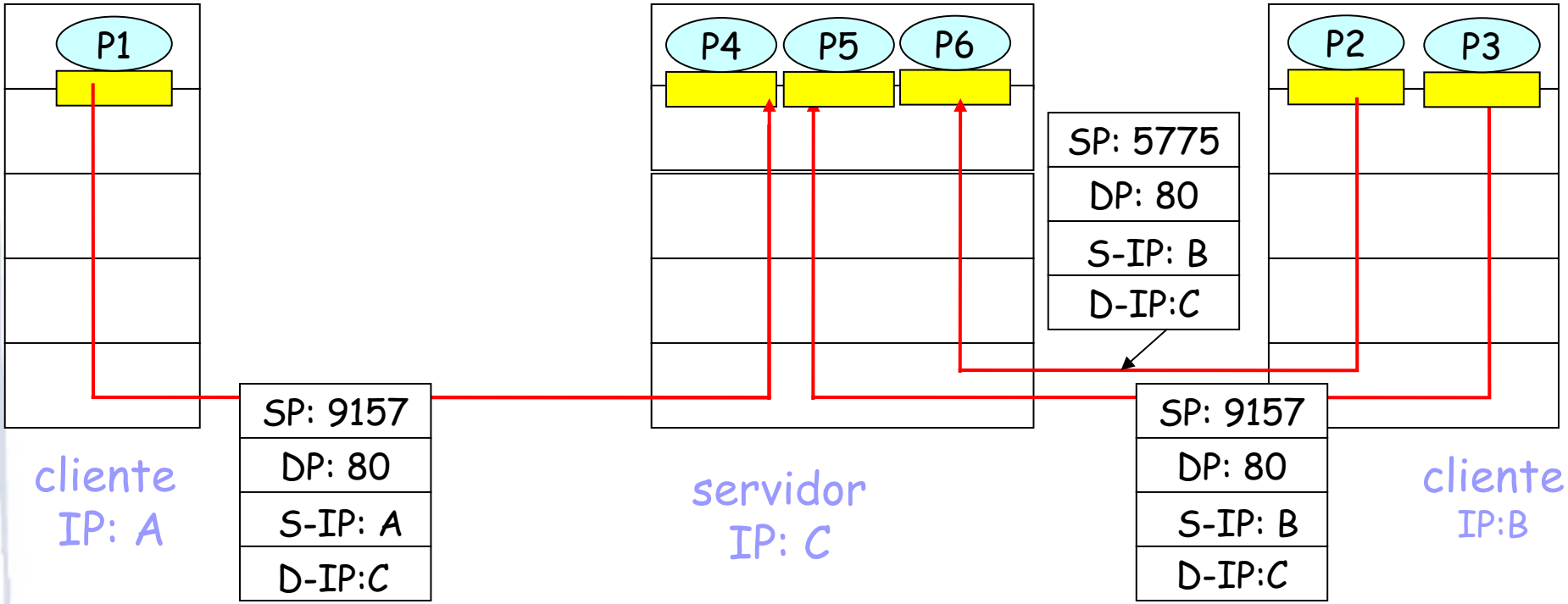
```
DatagramSocket serverSocket = new DatagramSocket(6428);
```



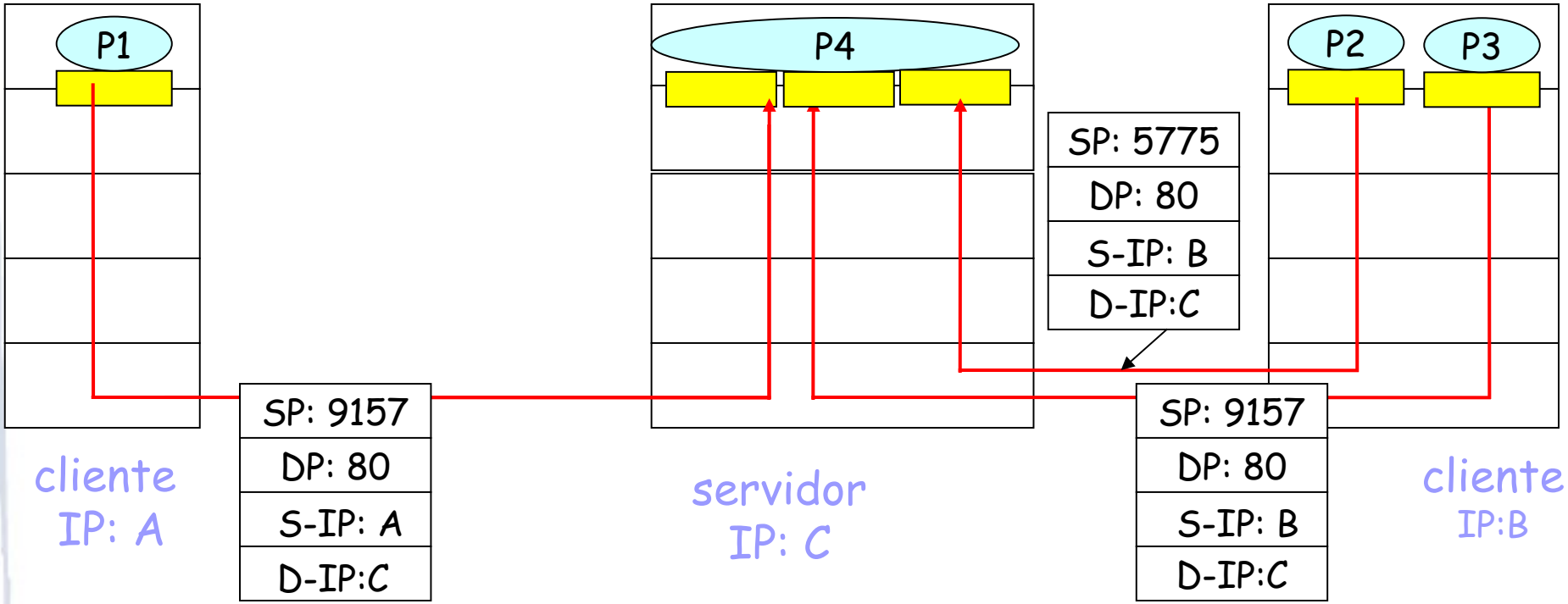
Demux orientada à conexão

- Socket TCP identificado por 4 valores:
  - Endereço IP de origem
  - End. porta de origem
  - Endereço IP de destino
  - End. porta de destino
- Hospedeiro receptor usa os quatro valores para direcionar o segmento ao socket apropriado
- Hospedeiro servidor pode suportar vários sockets TCP simultâneos:
  - Cada socket é identificado pelos seus próprios 4 valores
  - Servidores Web possuem sockets diferentes para cada cliente conectado
  - HTTP não persistente terá um socket diferente para cada requisição

Demux orientada à conexão



Demux orientada à conexão: Ex. servidor Web "*threaded*"



Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

UDP: User Datagram Protocol [RFC 768]

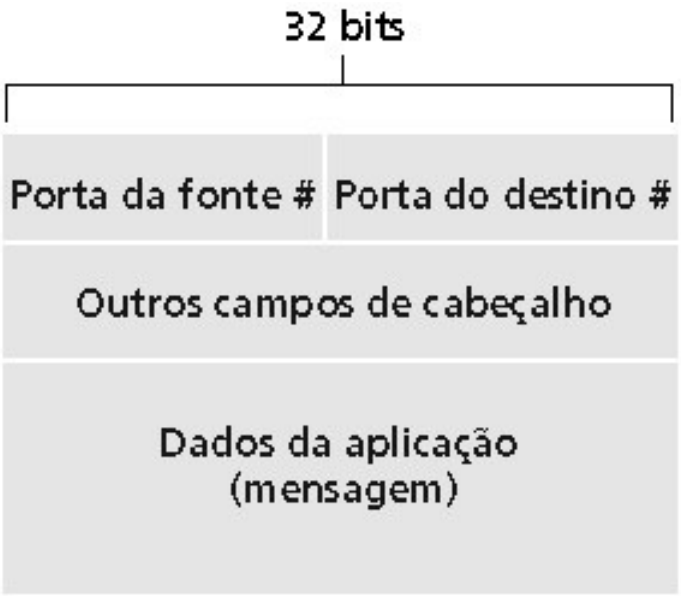
- Protocolo de transporte da Internet “sem gorduras” e “sem frescuras” :)
- **Serviço “best effort”**, segmentos UDP podem ser:
  - Perdidos
  - Entregues fora de ordem para a aplicação
- Sem conexão:
  - Não há apresentação entre o UDP transmissor e o receptor
  - Cada segmento UDP é tratado de forma independente dos outros

Por que existe o UDP?

- Não há estabelecimento de conexão (que possa redundar em atrasos)
- Simples: não há estado de conexão nem no transmissor, nem no receptor
- Cabeçalho de segmento reduzido
- Não há controle de congestionamento: UDP pode enviar segmentos tão rápido quanto desejado (**e possível**)

Mais sobre UDP

- Muito utilizado por aplicações de mutimídia contínua (*streaming*)
  - Tolerantes à perda
  - Sensíveis à taxa
- Outros usos do UDP:
  - DNS (consultas rápidas)
  - SNMP (gerenciamento mesmo em situações de congestionamento)
- Transferência confiável sobre UDP: acrescentar confiabilidade na camada de aplicação
  - Recuperação de erro específica de cada aplicação



UDP *checksum*

Objetivo: detectar “erros” (ex.: bits trocados) no segmento transmitido

Transmissor:

- Trata o conteúdo do segmento como seqüência de inteiros de 16 bits
- *Checksum*: soma (complemento de 1 da soma) do conteúdo do segmento
- Transmissor coloca o valor do *checksum* no campo de *checksum* do UDP

Receptor:

- Computa o *checksum* do segmento recebido
- Verifica se o *checksum* calculado é igual ao valor do campo *checksum*:
  - NÃO - erro detectado
  - SIM - não há erros. **Mas, talvez haja erros apesar disso**?

Exemplo: Internet *checksum*

- Note que, ao se adicionar números, um *vai um* do bit mais significativo deve ser somando ao resultado.
- Utilizar o complemento da soma final como o *checksum*.
  - Exemplo: adicione dois inteiros de 16 bits

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

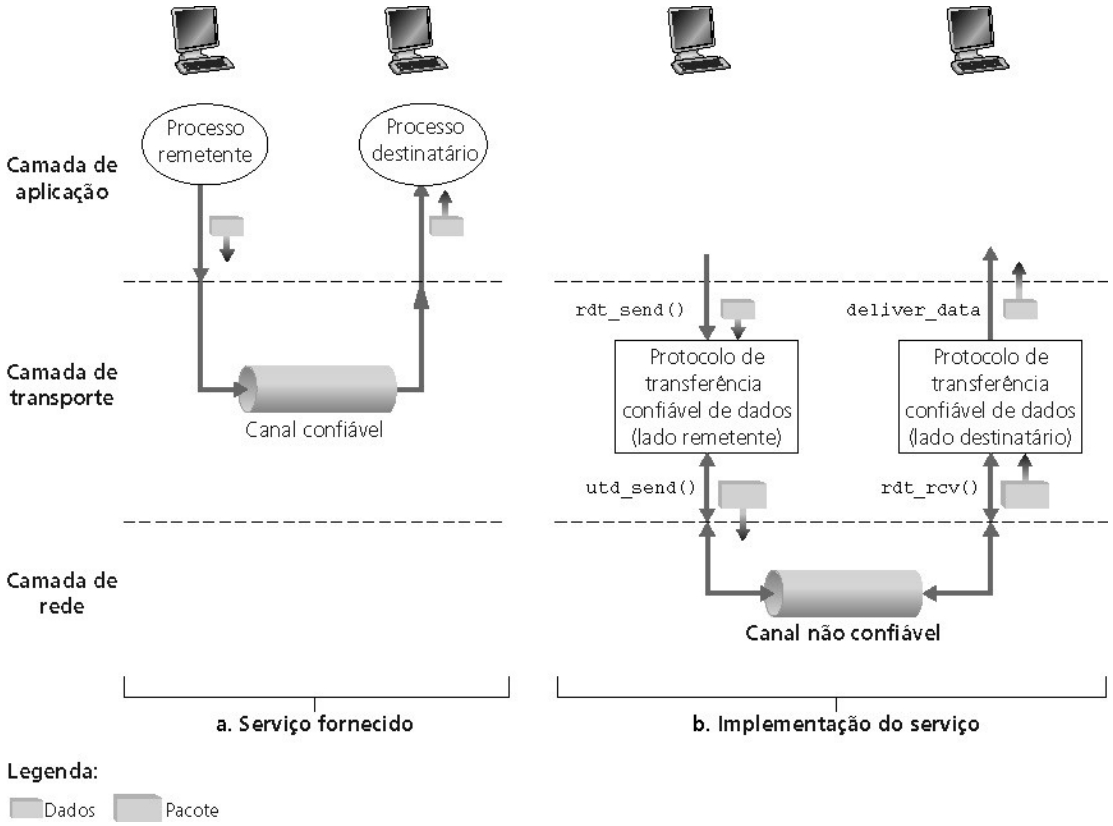
Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Princípios de transferência confiável de dados

- Importante nas camadas de aplicação, transporte e enlace

- Características dos canais não confiáveis determinarão a complexidade dos protocolos confiáveis de transferência de dados (rdt)



Canal com erros e/ou perdas

- Canal subjacente pode trocar valores dos bits num pacote
  - *Checksum* para detectar erros de bits
- A questão: como recuperar esses erros:
  - **Reconhecimentos (ACKs):** receptor avisa explicitamente ao transmissor que o pacote foi recebido corretamente
  - **Reconhecimentos negativos (NACKs):** receptor avisa explicitamente ao transmissor que o pacote tem erros
  - Transmissor reenvia o pacote quando da recepção de um NACK

Canal com erros e/ou perdas

- Canal de transmissão pode também **perder pacotes** (dados ou ACKs)
- Transmissor espera um tempo “razoável” (i.e., *timeout*) pelo ACK
- Retransmite se nenhum ACK for recebido nesse tempo

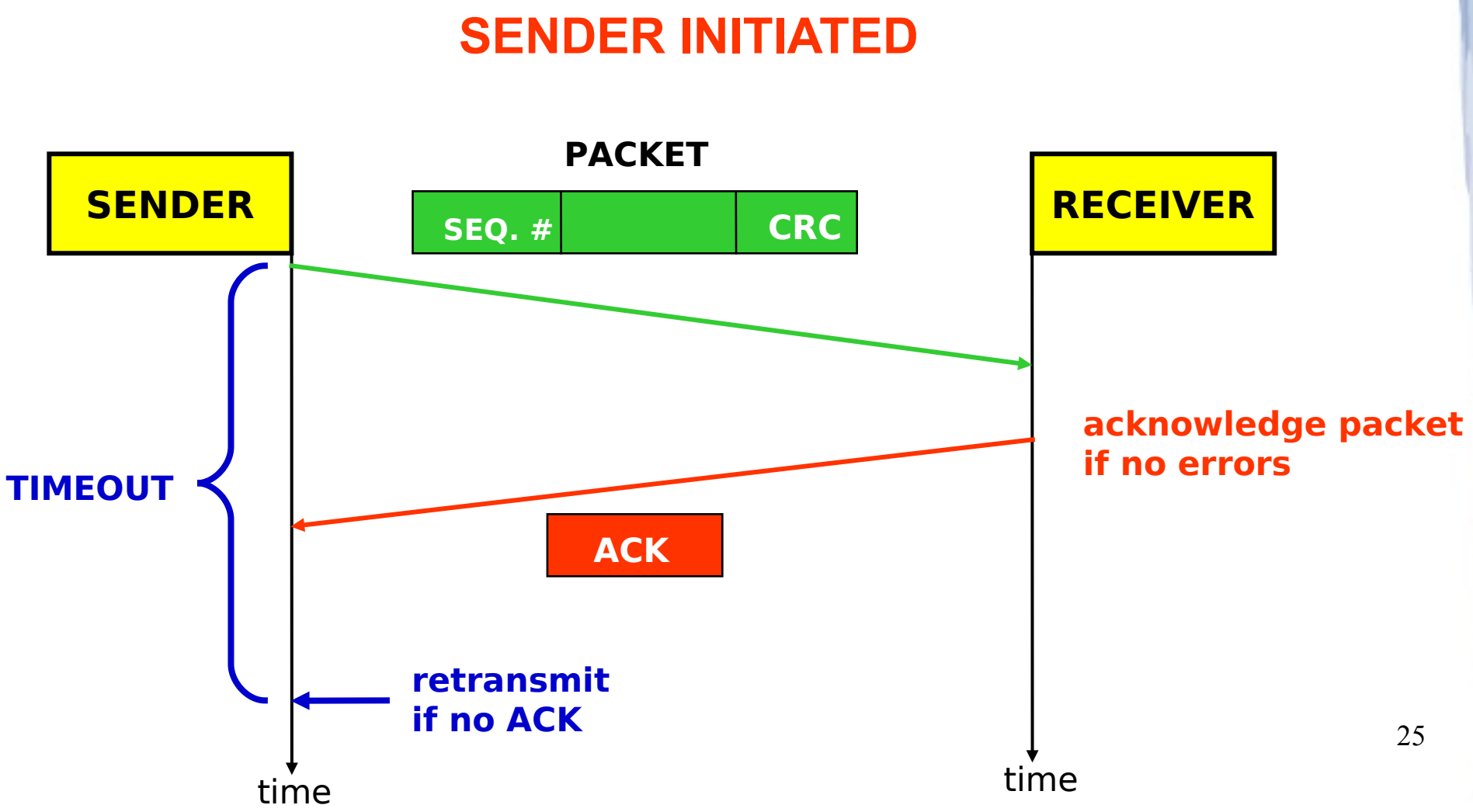
Estratégias de Retransmissão

- Conhecidos como algoritmos ou protocolos ***Automatic Repeat Request*** (ARQ).
- Questões de projeto:
  - Como o receptor requisita uma retransmissão?
  - Como a fonte sabe quando retransmitir?
- Desempenho e exatidão.
- Para simplificar as explicações assumiremos enlaces do tipo ponto-a-ponto.

ARQ: hipóteses

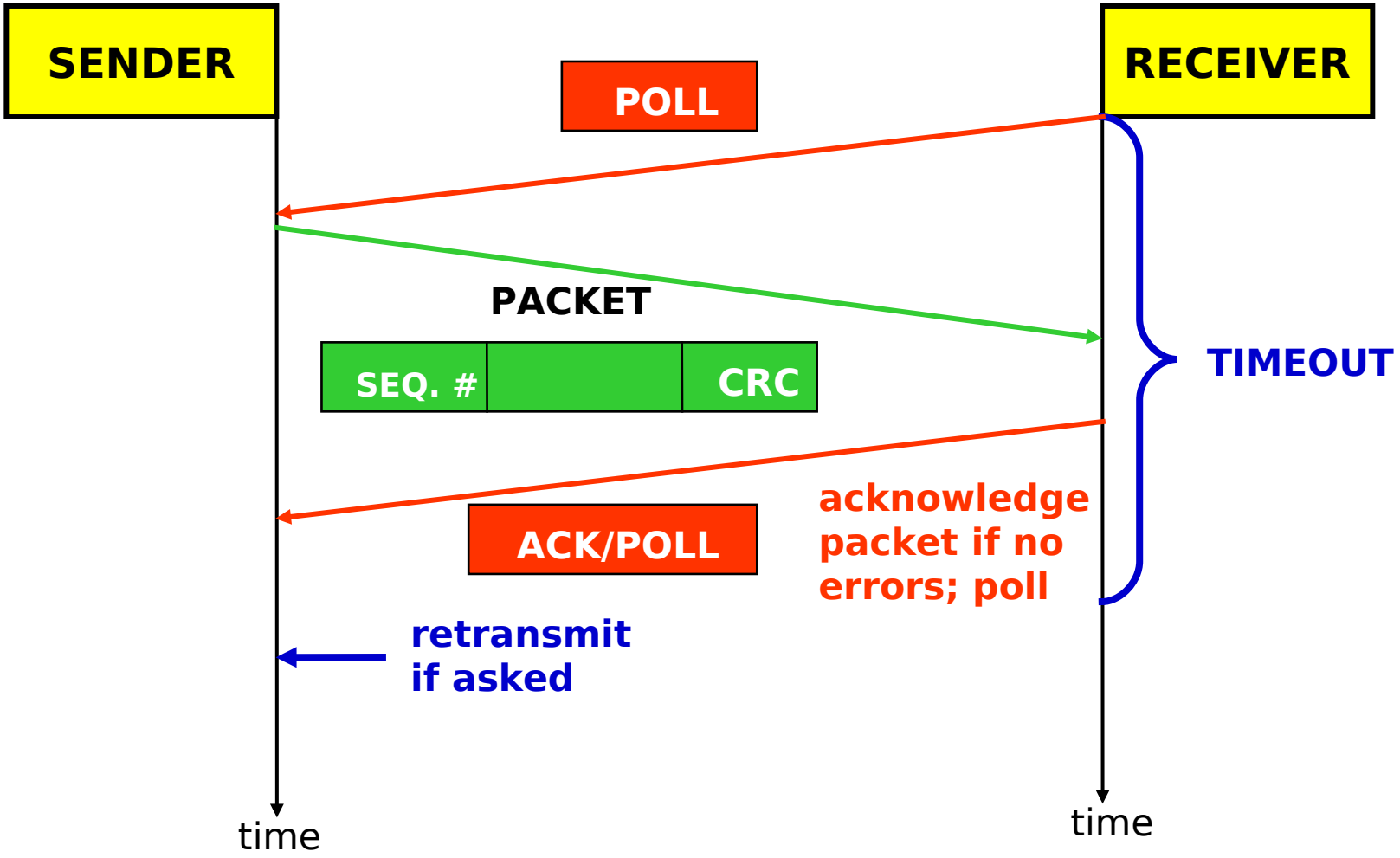
- Enlaces do tipo ponto-a-ponto.
- Uma fonte e um destino.
- Enlaces e nós seguem a ordem FIFO.
- Os nós executam o protocolo ARQ corretamente.
- A sessão entre fonte e destino já está inicializada e é permanente.
- Todos os erros são detectados corretamente e o enquadramento é perfeito.

Esquema ARQ genérico em canais FIFO



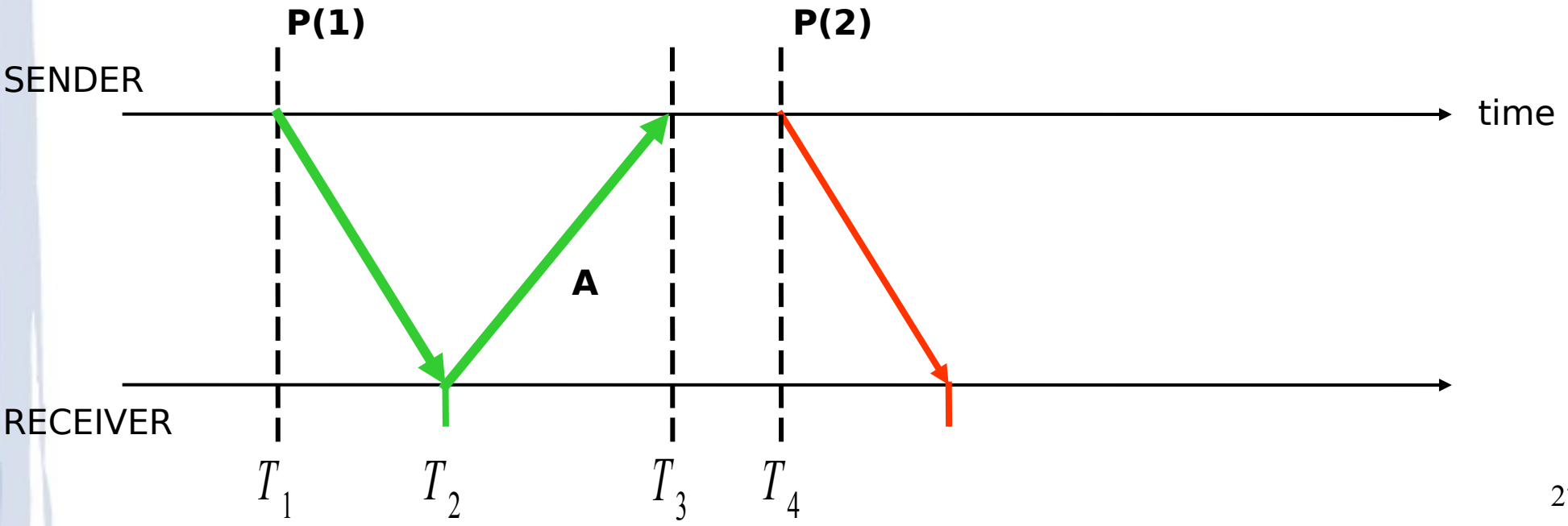
Esquema ARQ genérico em canais FIFO

RECEIVER INITIATED



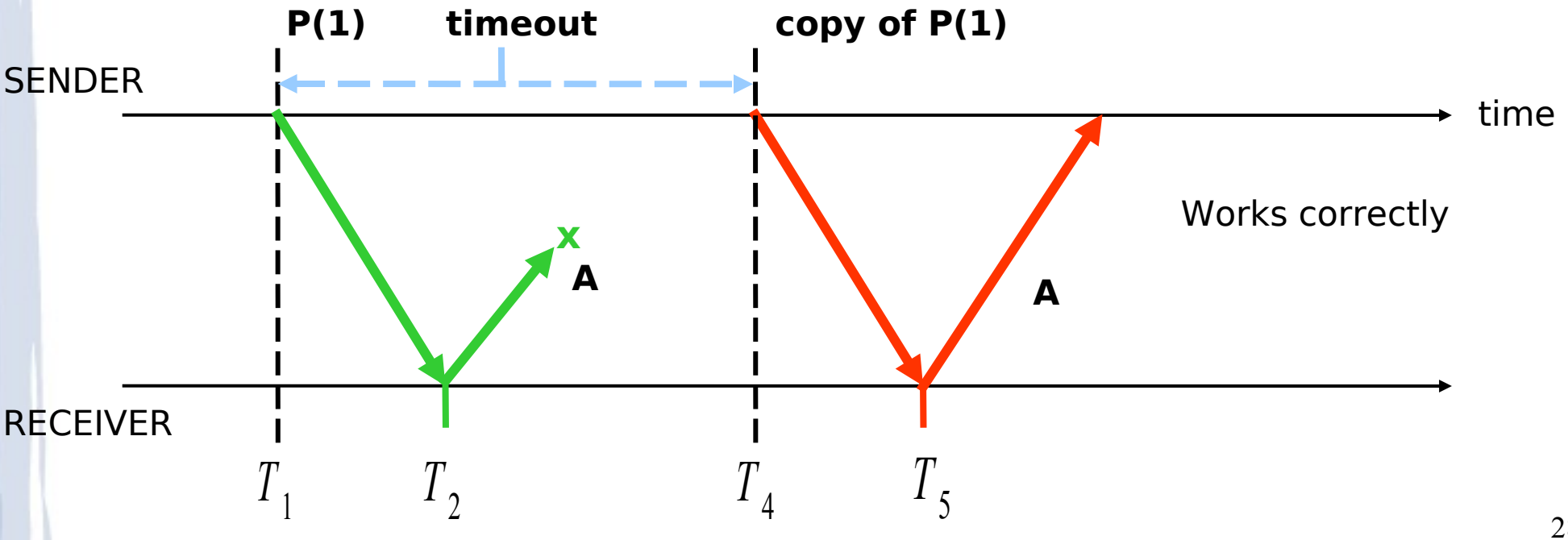
Exemplo de um ARQ simples

- Assuma que o destino rotule toda confirmação (ACK) com o mesmo rótulo “A” independentemente de qual pacote esteja sendo reconhecido.

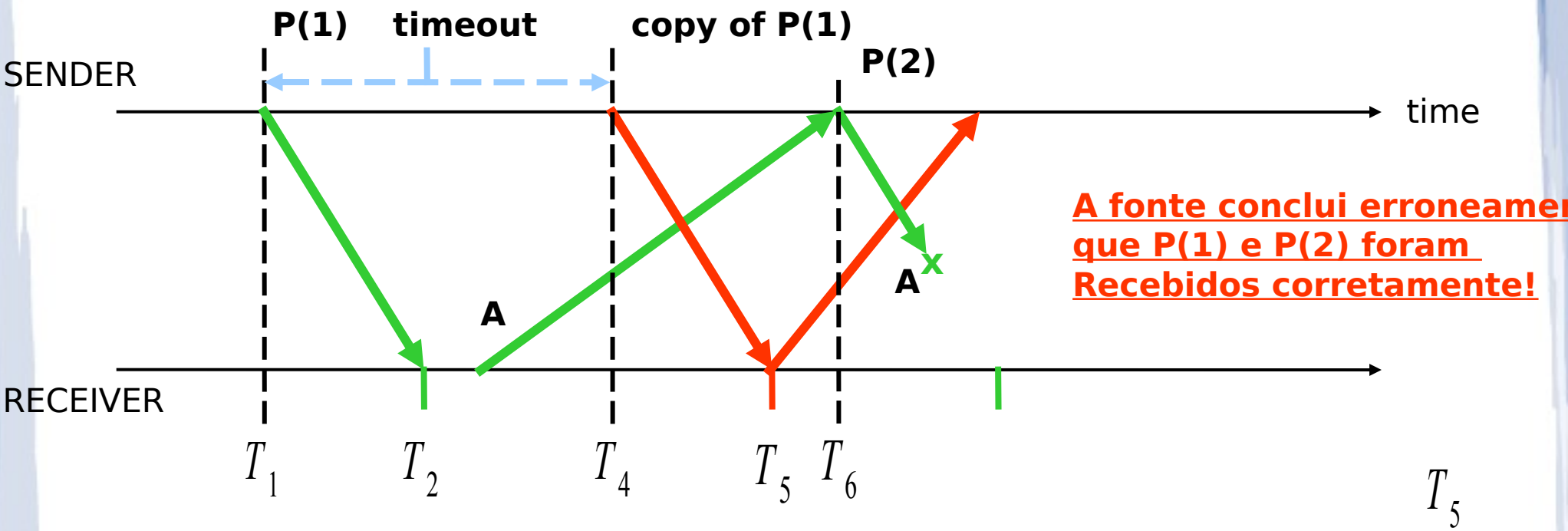


Erros podem confundir o destino

- Assuma que a fonte retransmite um pacote quando não recebe um ACK dentro de um determinado tempo fixo (i.e., ***timeout***).



Erros podem confundir o destino



Para evitar confusão na fonte, cada ACK deve referenciar qual pacote está sendo reconhecido!

Requisitos em ARQ

- A fonte rotula cada pacote enviado utilizando um espaço de **numeração sequencial**.
- O destino envia um ACK para cada pacote que ele recebe sem erros e numera cada ACK com o número sequencial do pacote correspondente.
- Caso expire o tempo de espera na fonte sem receber a confirmação da recepção pelo destino isto resulta na retransmissão do pacote pela fonte.

ARQ do tipo *Stop-and-Wait*

- Fonte transmite pacotes rotulados 1, 2,….. (ou apenas 1 bit: valores 0 ou 1)
- Destino envia ACK para todo pacote recebido corretamente sendo que o ACK especifica o <u>próximo pacote esperado</u>.
- Destino passa cópia do pacote recebido corretamente para o nível de rede descartando pacotes com erro.
- Fonte retransmite pacote sem confirmação após um tempo fixo de espera (i.e., *timeout*).
- Fonte e destino são incializados para enviar e receber pacote com número de sequência 0 ou 1.

Stop-and-Wait Protocol (SWP)

Sender's algorithm:

1. Initialize packet number (PN) to 0 [or 1 for example].
2. Wait until packet is received from network layer; assign PN to packet.
3. Transmit (or retransmit) packet in frame with PN as its sequence number.
4. Start timeout (TO) timer and wait for ACK from receiver.
5. If error-free ACK specifying PN+1 is received within TO interval,

   then set PN = PN+1 and go to Step 2.

   else go to Step 3

Stop-and-Wait Protocol

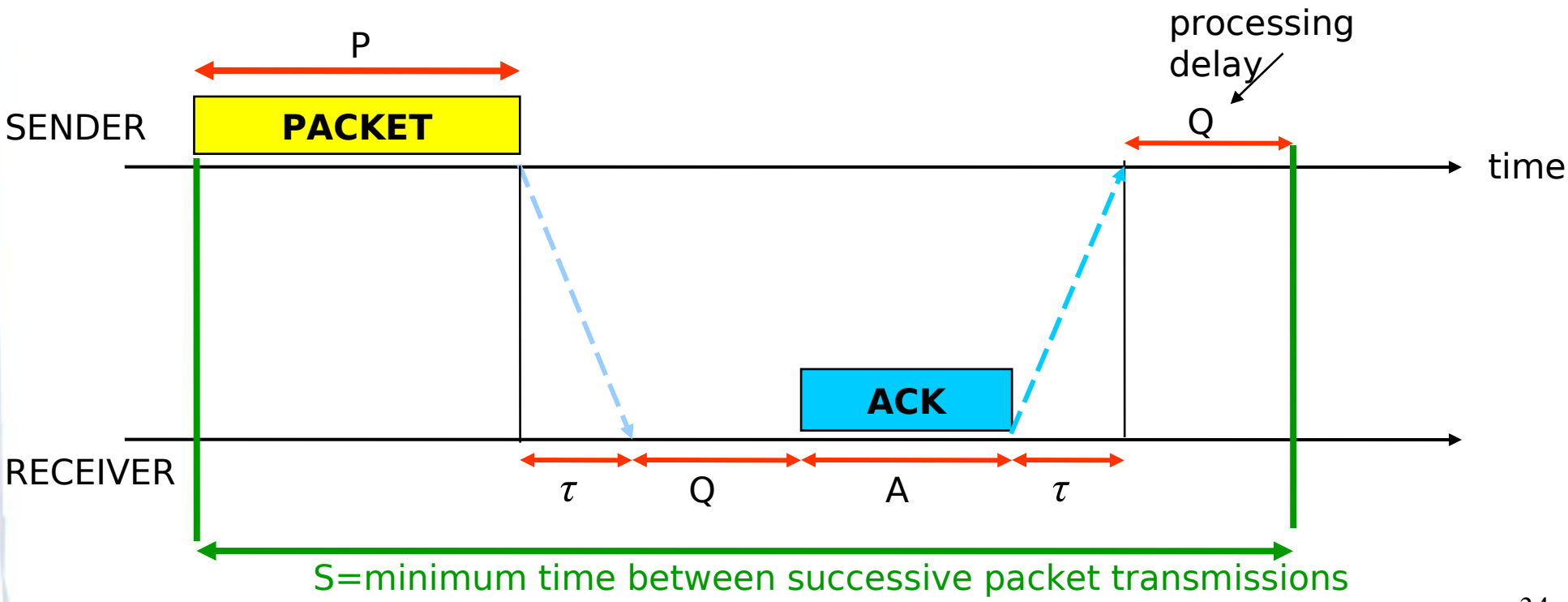
Receiver's algorithm:

1. Initialize next packet (NP) to 0 [or 1 for example] and repeat Step 2 forever
2. If (packet is received)

```
then
if (error in packet)
   then discard packet
   else do
        if ( PN=NP )
           then give packet to network layer and set
NP = NP+1
        send ACK specifying NP
```

Eficiência do SWP

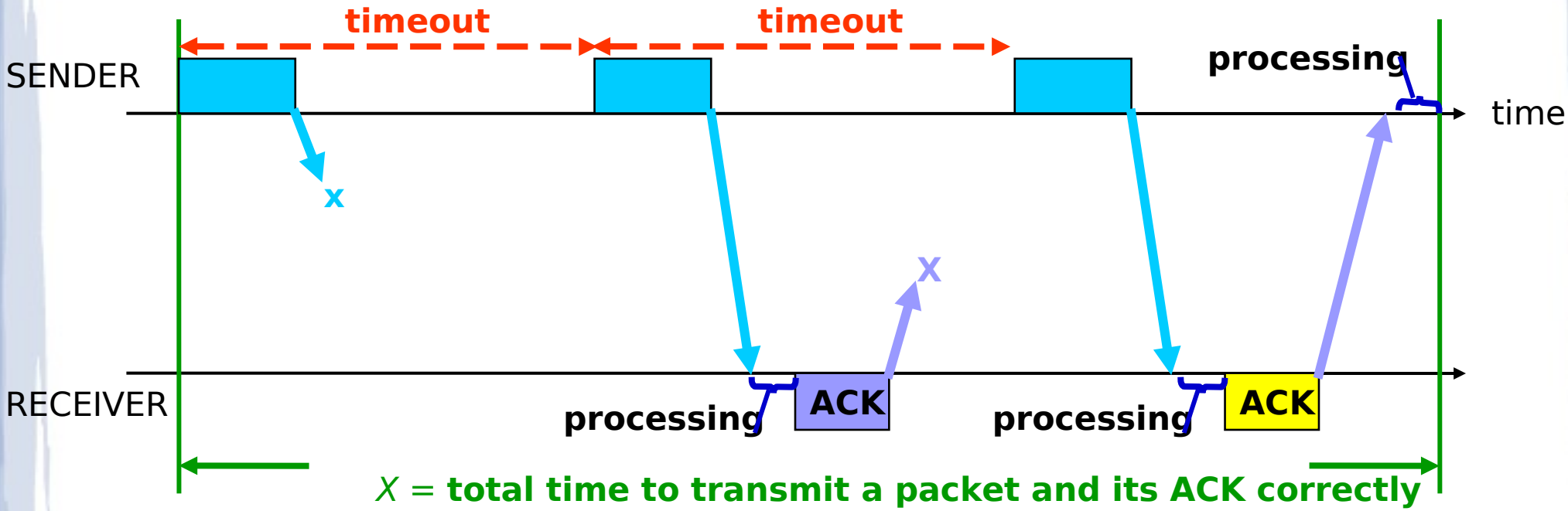
- Considere o caso sem erros:



Eficiência do SWP

Do diagrama anterior temos que: $S = P + A + 2(Q + \tau)$

Entretanto, pacotes de dados ou ACKs podem ser perdidos ou corrompidos no canal resultando em um tempo mais prolongado para entregar o pacote ao destino.

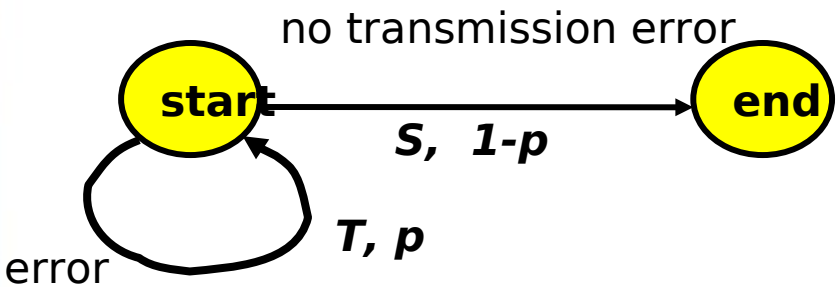


Eficiência de SWP

- Vamos assumir que *p* é a probabilidade de um pacote de dados, ou o seu ACK, ser enviado com erro.
- Isto implica que cada tentativa de enviar um pacote com sucesso é independente de outras tentativas.
  - Variável randômica geométrica!
- O número médio de transmissões necessárias para enviar uma mensagem com sucesso é $1/(1 - p)$.
- Todas as transmissões falhas acarretam um tempo equivalente a um *timeout* e isto acontece com probabilidade p.
- A última transmissão acarreta S segundos e acontece com probabilidade $(1 - p)$.

Eficiência do SWP

- Podemos aplicar um método gráfico simples:



T = timeout
S = P+A+2Q+2τ

$$E\{X\}=(1-p)S+p(T+E\{X\})$$

$$E\{X\}=S+\frac{pT}{1-p}$$

$$Efficiency=\frac{packettime}{E\{X\}}=\frac{P}{E\{X\}}$$

$$\eta=\frac{(1-p)P}{(1-p)S+pT}$$

Muito overhead em enlaces com grande *bandwidth-delay product*!

$$\eta\approx\frac{(1-p)P}{(1-p)(P+A+2\tau)+pT};\quad Q=0$$

Eficiência do SWP
(um exemplo mais intuitivo)

- Exemplo: enlace de 1.5 Mbps com atraso fim-a-fim (i.e., RTT) de 45 ms – o que dá um *bandwidth X delay* de 67.5 Kb (~ 8 KB) (assumindo full duplex!)
- A fonte pode enviar somente um pacote por RTT.
- Assumindo pacotes de 1 KB, isto implica uma taxa máxima de transmissão de
  - 1024 x 8 / 0.045 = 182 Kbps
  - Ou seja, aproximadamente 1/8 da capacidade do enlace!

Repetição Seletiva

- ***Motivação:*** SWP deixa a fonte ociosa por períodos longos esperando por ACKs.
- ***Solução:*** Permitir que a fonte transmita múltiplos pacotes enquanto espera por confirmações. ***Have a pipeline of packets!***

Repetição Seletiva

- ***Requisitos:***
  - Fonte e destino podem armazenar ***W*** pacotes
  - A fonte rotula os pacotes utilizando números consecutivos 1, 2,….
  - O destino armazena os pacotes recebidos sem erros, confirma os mesmos com ACKs, e os entrega ordenados ao nível superior.

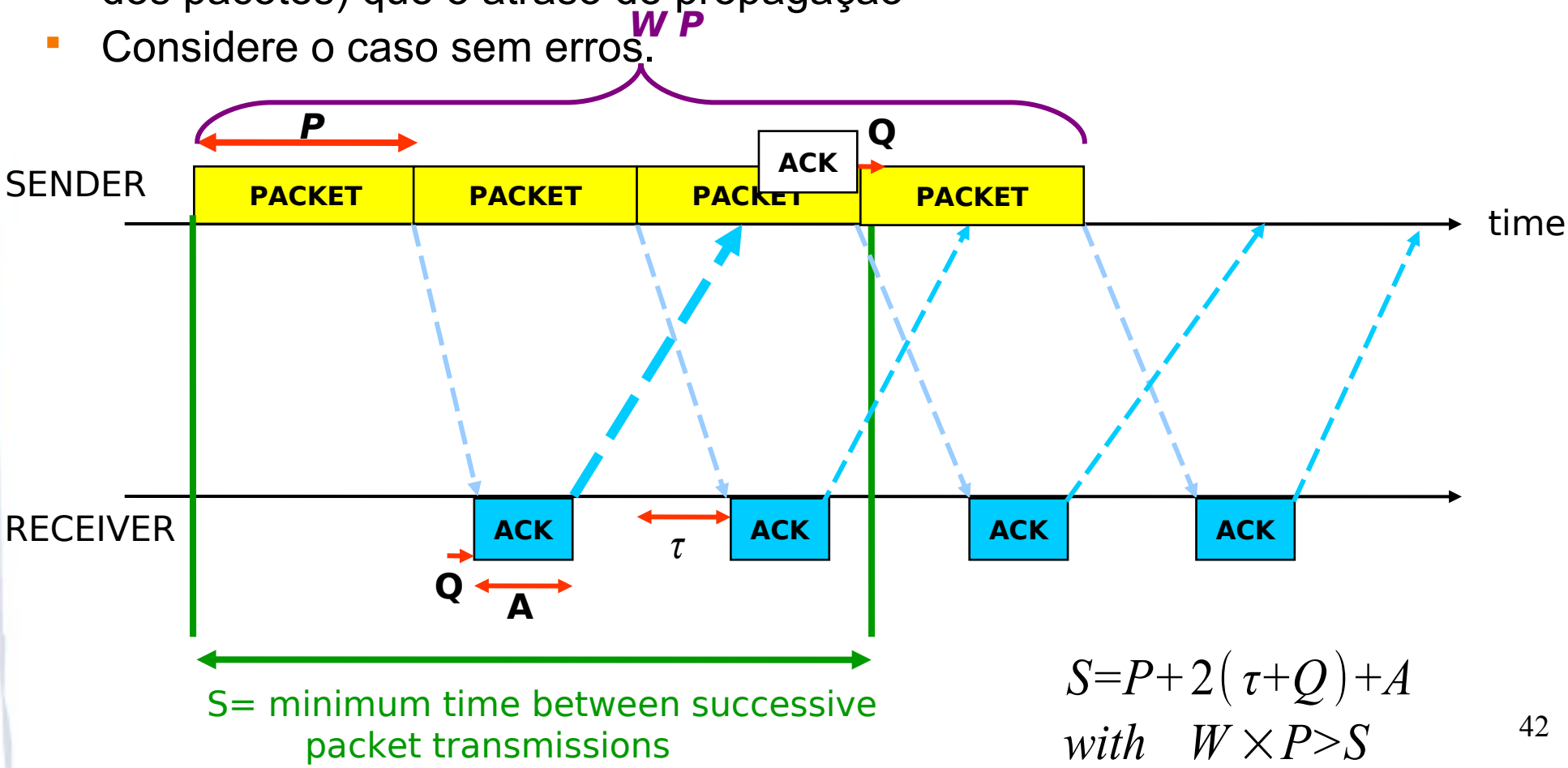
    (e.g., se o pacote P1 tem erros e P2 e P3 estão corretos, o destino segura esses até receber P1 corretamente)
  - A fonte mantém cópias dos pacotes transmitidos até que ela receba confirmação da recepção dos mesmos.
  - A fonte retransmite um pacote quando o ***timeout*** do mesmo expira sem ter recebido ACK
  - **ACK referencia o número de sequência do pacote sendo confirmado**

Repetição Seletiva

- ***Diversas variantes possíveis, assumiremos a seguinte:***
  - Pacote **ACK** como descrito anteriormente (confirmação de um determinado pacote com número de sequência *n*)
  - Pacote **ACK** também especifica qual o próximo pacote *NP* esperado pelo destino (*i.e.*, qual o próximo pacote na sequência esperado pelo destino)
    - **Todos os pacotes no intervalo [*NP – W, NP – 1*] estão automaticamente confirmados. Por exemplo, com W=4 e números de sequência de 0 a 7, quando NP=2 significa que as mensagens com números de sequência 6, 7, 0 e 1 foram recebidas pelo destino.**

Eficiência do SRP

- Tipicamente os pacotes são mais longos (i.e., tempo de transmissão dos pacotes) que o atraso de propagação
- Considere o caso sem erros.

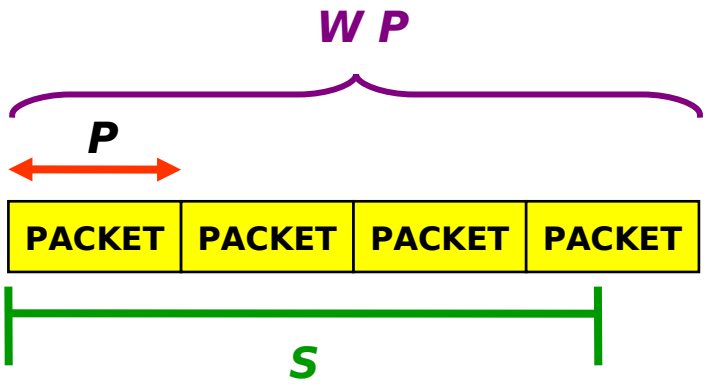


$$S = P + 2(\tau + Q) + A$$

$$with \quad W \times P > S$$

Eficiência do SRP

- Eficiência do SRP: $$\eta_{SRP} = \min\left\{\frac{WP}{S}, 1\right\}$$



Sem erros, a fonte nunca pára de transmitir com ***S < WP***

Ou seja, pacotes podem ser (r)enviados continuamente porque confirmações estão chegando!!!

Assuma ***W infinito*** e ***T > S***, de tal forma que a fonte nunca retransmite caso não tenha erros.

p *= P{erro de transmissão no pacote de dados ou ACK}*

1-p *=P{transmissão com sucesso}*

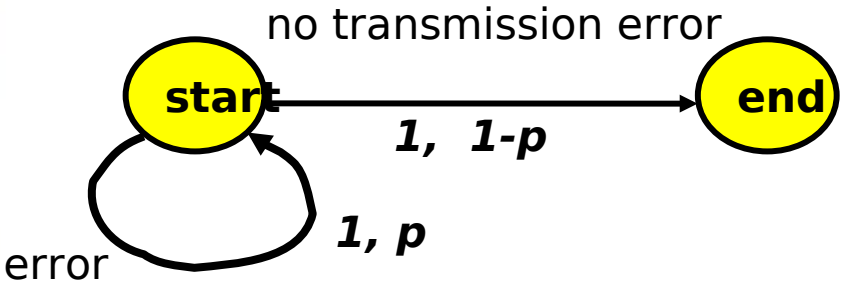
Y= número de vezes que um pacote é transmitido

O número médio de transmissões para ter sucesso (distribuição geométrica: E(Y)=1/P{sucesso}):

$$E(Y) = \frac{1}{1-p}$$

Eficiência do SRP

- Também podemos utilizar o método gráfico…
- Concentremos no número de transmissões porque todas as transmissões tem a mesma duração (*i.e.*, ***assumindo que a fonte nunca pára de transmitir***).



$$E\{Y\} = (1-p) + p(1 + E\{Y\})$$

$$E\{Y\} = \frac{1}{1-p}, \quad \text{como esperado!}$$

E(*X*) *é o tempo médio para transmitir um pacote com sucesso*, então *E(X)= P E(Y)*

$$Efficiency = \frac{packettime}{E\{X\}} = \frac{P}{E\{X\}}$$

$$\eta_{SRP} = \frac{P}{P/(1-p)} = 1-p \quad \text{for } W = \infty$$

Muito eficiente, desde que a janela seja mais longa do que S !

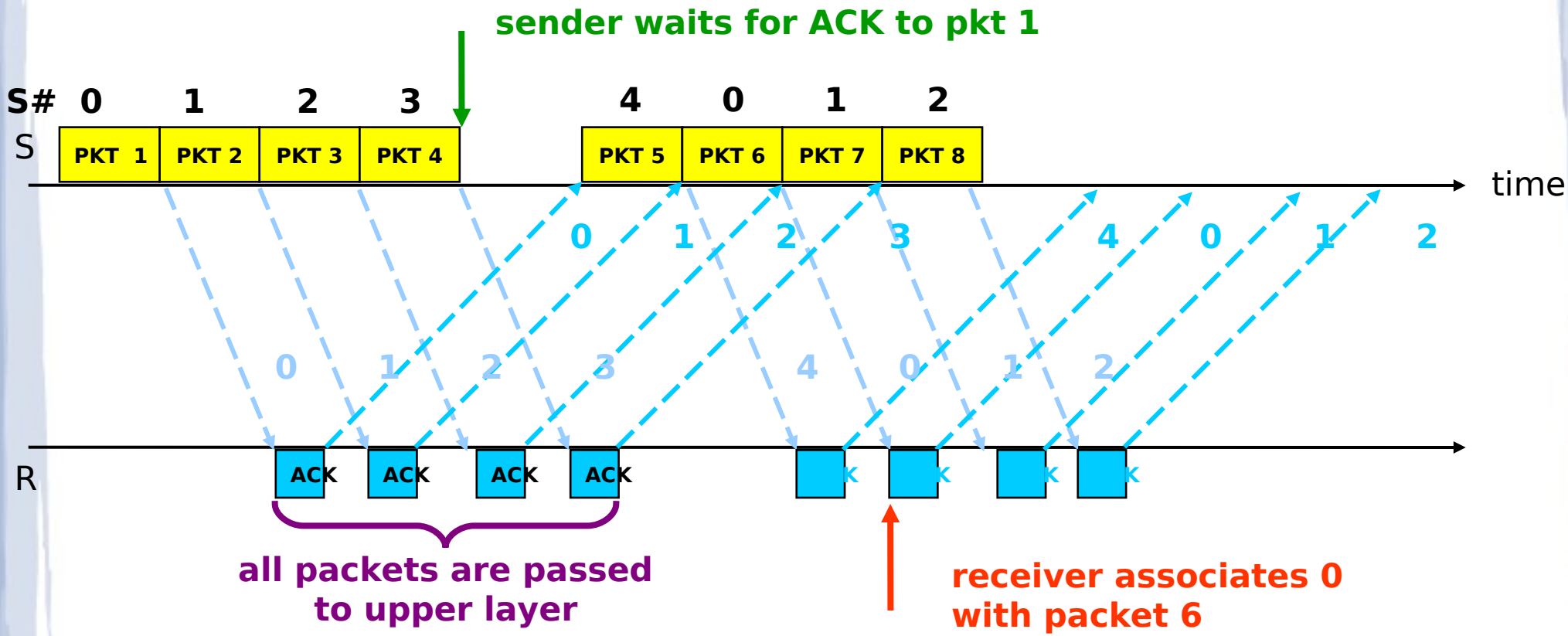
Resultado intuitivo! **Caso a fonte nunca pare, a eficiência é a % de pacotes enviados sem erro!**

Correção de Repetição Seletiva

- O que provar:
  - ***Espaço finito de números de sequência:*** Qual é dimensão mínima do espaço de números de sequência de forma que a fonte e o destino nunca se confundam (*i.e.*, número de sequência no **ACK** e pacotes).
    - *Deve-se assegurar que após o receptor ter ajustado a sua janela não haja nenhuma sobreposição com a janela original!!!*
  - ***Safety:*** O destino passa os pacotes em sequência para o nível superior, sem intervalos ou réplicas.
  - ***Liveness:*** *Deadlocks* nunca acontecem.

Números de sequência em SRP

- Assuma W=4 e 5 números de sequência (i.e., 0 a 4)

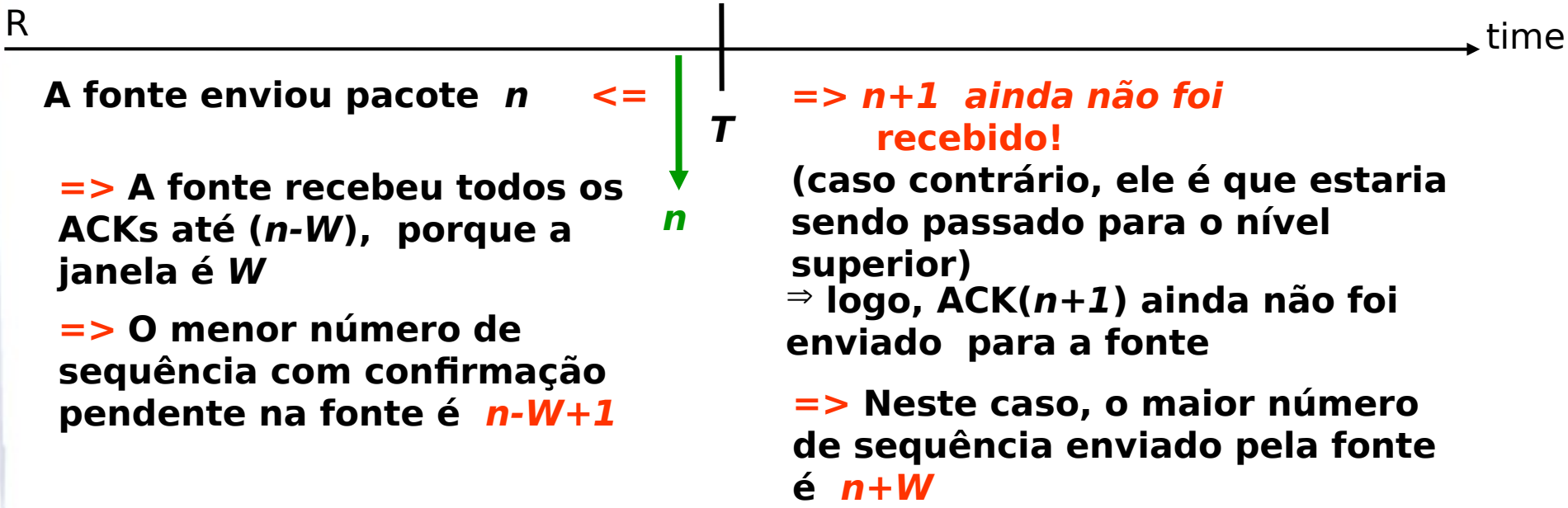


Números de sequência no SRP

- **W>(#s de sequência)/2 pode confundir o receptor**
- Exemplo: considere W=3 e #Seq=4 (i.e., 0..3)
- Fonte envia pacotes com #seq 0, 1 e 2
- Assuma que o receptor recebe todos os pacotes e envia confirmações ***mas todas as confirmações são perdidas***
- Neste ponto, o ***receptor avançou sua janela de recepção*** para os #seq 3, 0 e 1
- A fonte não recebendo confirmações vai retransmitir os pacotes #seq 0, 1 e 2
- O receptor vai aceitar os pacotes com #seq 0 e 1 porque está dentro da janela de recepção atual! ***Erroneamente aceita pacotes repetidos como pacotes novos!***
- Confirma a recepção de 0 e 1.
- A fonte recebendo ACK para #seq 0 e 1 avança a sua janela para #seq 2, 3 e 0...

Números de sequência em SRP

- Assuma janela de tamanho *W* e pacotes são numerados em módulo *2W* (i.e., de **0 a *2W* -1**)
- Assuma que no tempo *T* o pacote rotulado ***n*** seja passado para o nível superior no destino (e é o pacote com o maior número que pode ser passado para o nível superior; *i.e.*, em sequência!).



Dado n, a faixa de números de sequência possíveis na fonte é *{n-W+1, n+W}* e utilizando o espaço *modulo 2W* para os números de sequência funciona corretamente!

Outra perspectiva para a análise da numeração e tamanho máximo da janela:

Assumindo janela de tamanho W. O pior caso é quando o transmissor não atualizou sua janela (i.e., não houve nenhum deslocamento dela). No entanto, vamos assumir que o receptor atualizou por completo sua janela (i.e., deslocou W posições) e **todas** as confirmações foram perdidas (motivo pelo qual a fonte não atualizou a sua janela).

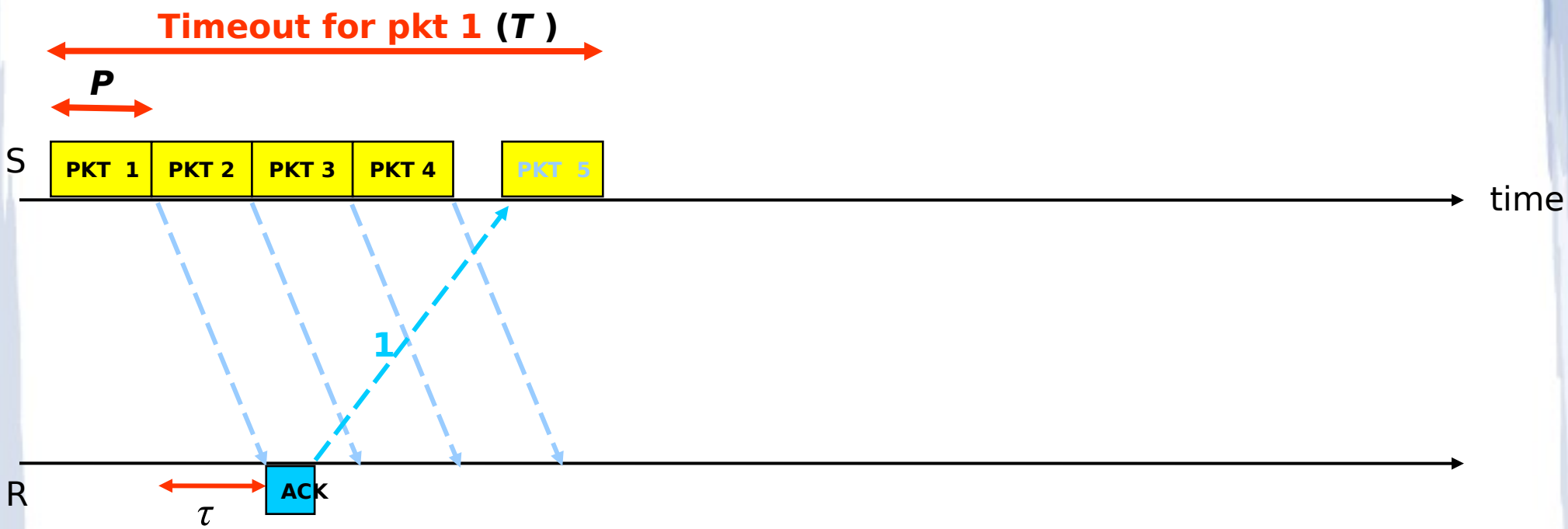
Para que não ocorra sobreposição, os valores dos números de sequência presentes na janela do transmissor $\mathbf{W_T}$ devem ser distintos dos números de sequência na janela atualizada do receptor $\mathbf{W_R}$. Portanto, necessita-se, no mínimo, **2W** números de sequência distintos.

Isso é análogo a definir o tamanho máximo da janela quando se fixa/limita a quantidade de números de sequência :)

Go-Back-N (GBN) ARQ

- Repetição seletiva exige que a fonte e o destino tenham um *buffer* (**o que não é um problema hoje em dia!**).
- Com GBN, o destino descarta qualquer pacote fora de ordem; portanto, não necessita de um *buffer.*
- Destino confirma (i.e., ACK) um pacote recebido corretamente com o número de sequência do último pacote recebido em ordem.
- A fonte inicializa um tempo de espera para cada pacote transmitido. Caso não receba confirmação dentro deste tempo, a fonte retransmite o pacote expirado ***e todos os pacotes enviados após aquele pacote.***
- A fonte pode ter até ***W*** pacotes esperando por confirmação.

GBN ARQ Exemplo 1

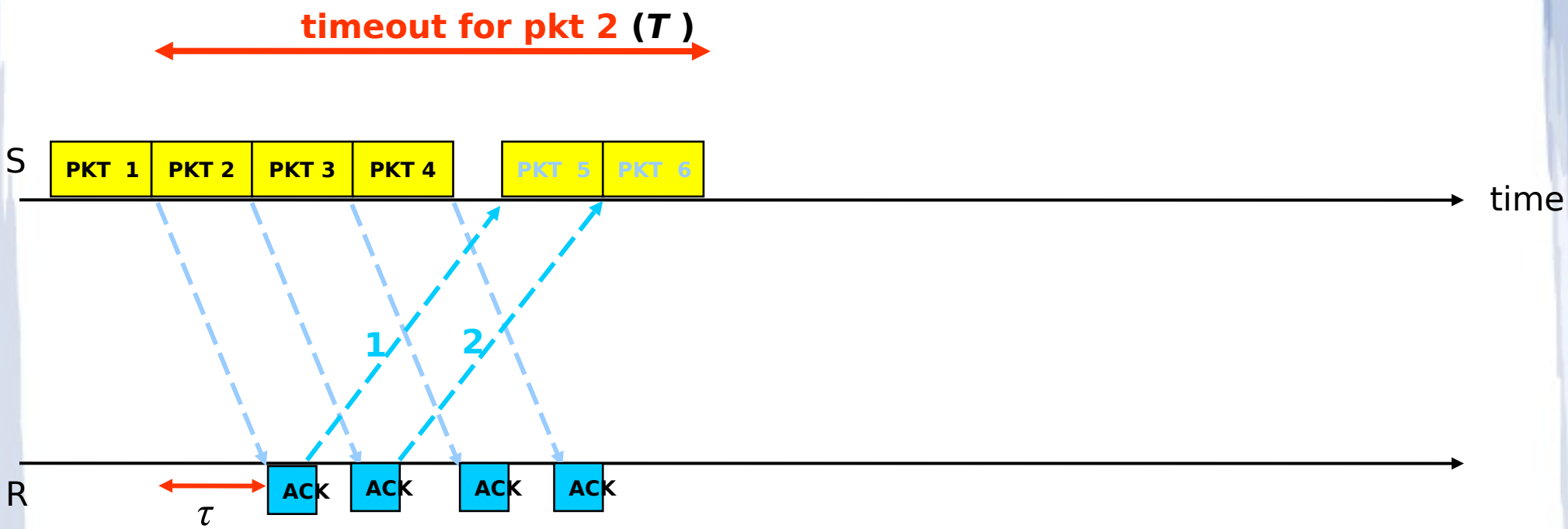


Packet 1 is received correctly; receiver sends ACK

Sender is able to transmit four packets before timeout for pkt 1 expires

ACK to pkt 1 arrives before its timeout expires; sender can send packet 5.

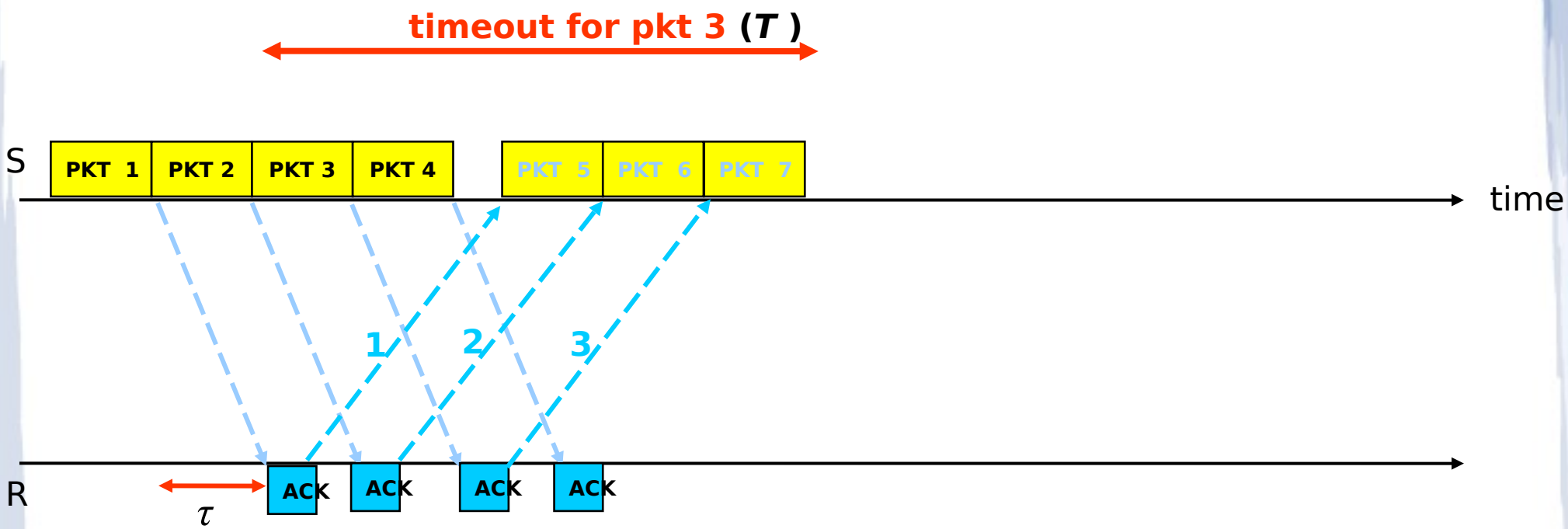
GBN ARQ Exemplo 1



Packets 2 to 4 arrive in order at the receiver; receiver sends ACKs for them

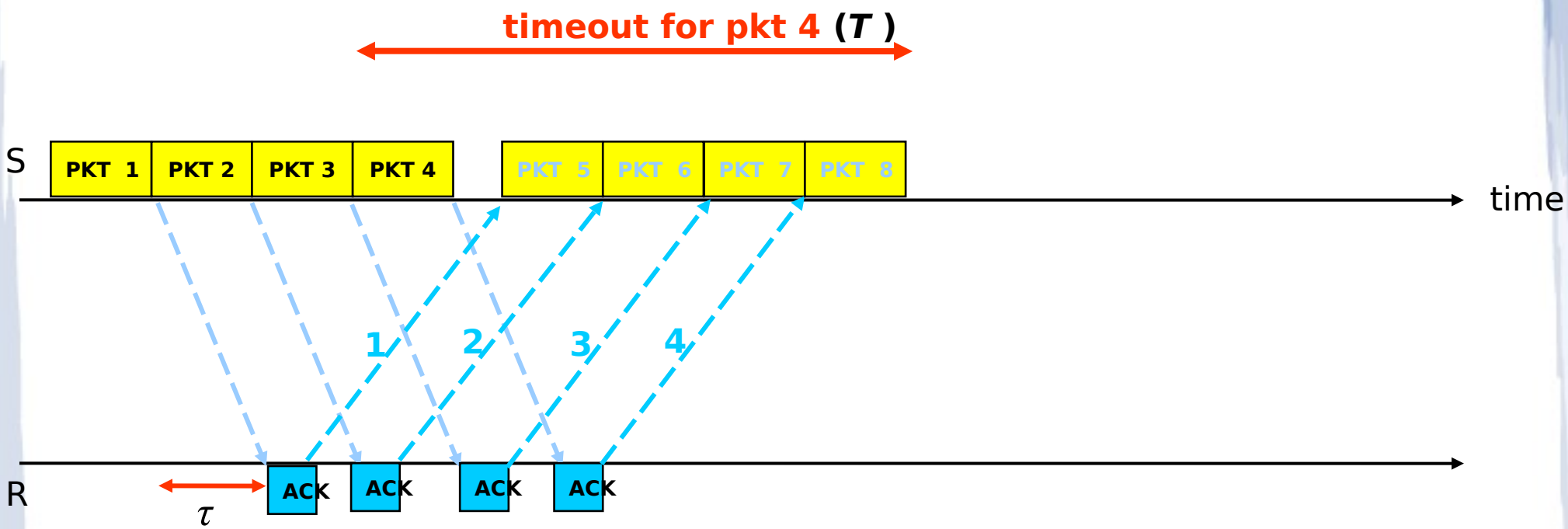
ACK to pkt 2 arrives before its timeout expires; sender can send packet 6.

GBN ARQ Exemplo 1



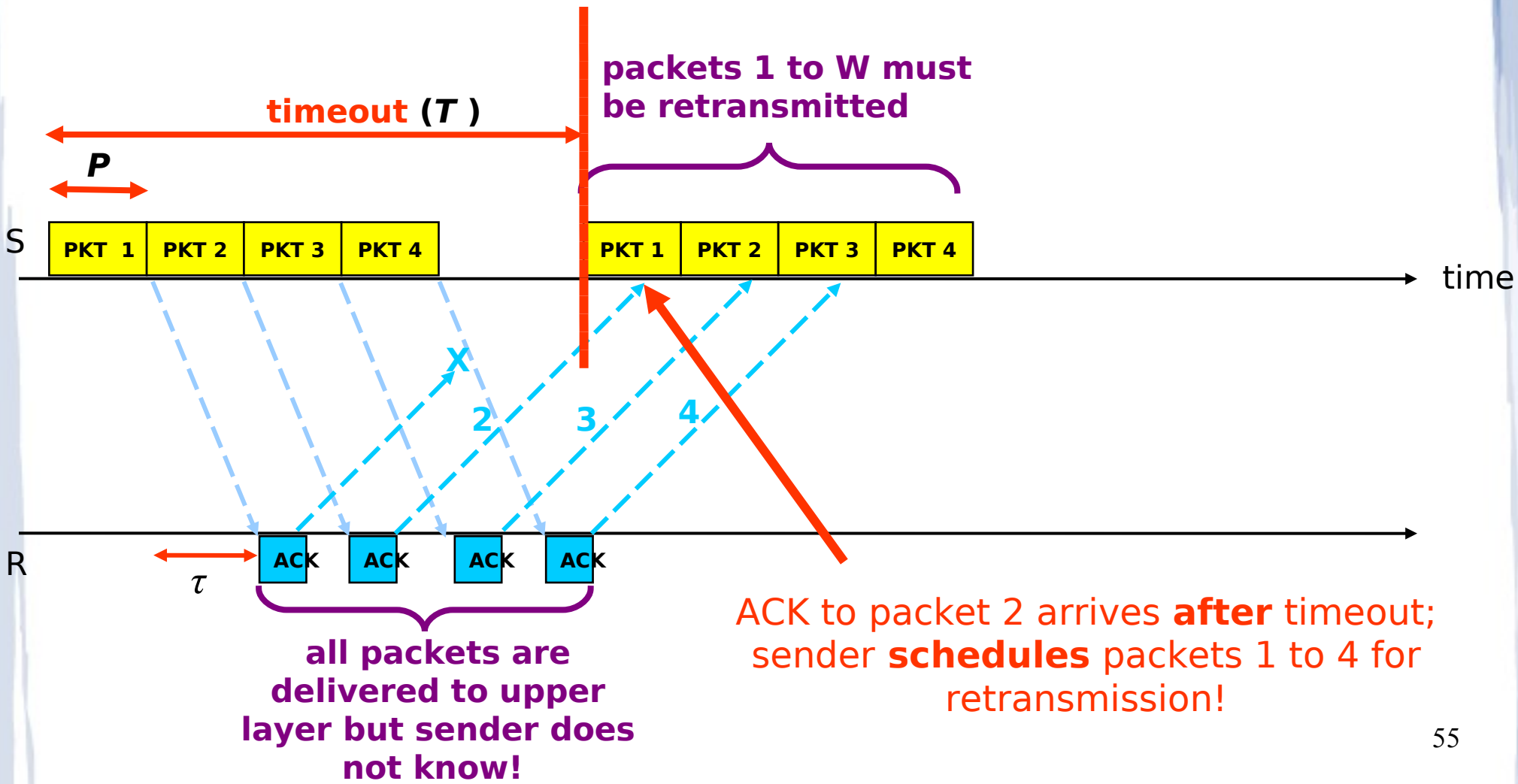
ACK to pkt 3 arrives before its timeout expires; sender can send packet 7.

GBN ARQ Exemplo 1

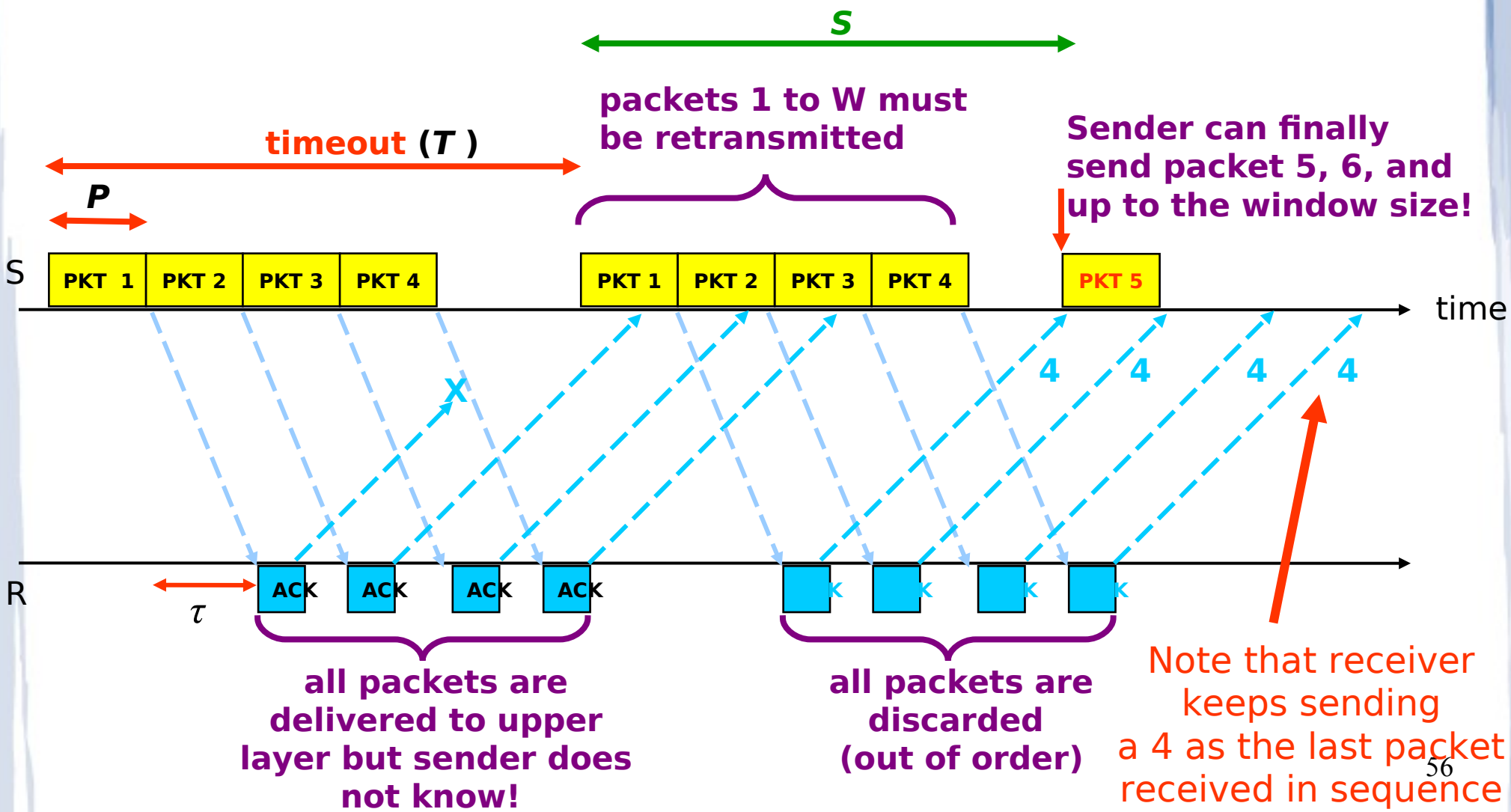


ACK to pkt4 arrives before its timeout expires; sender can send packet 8, etc.

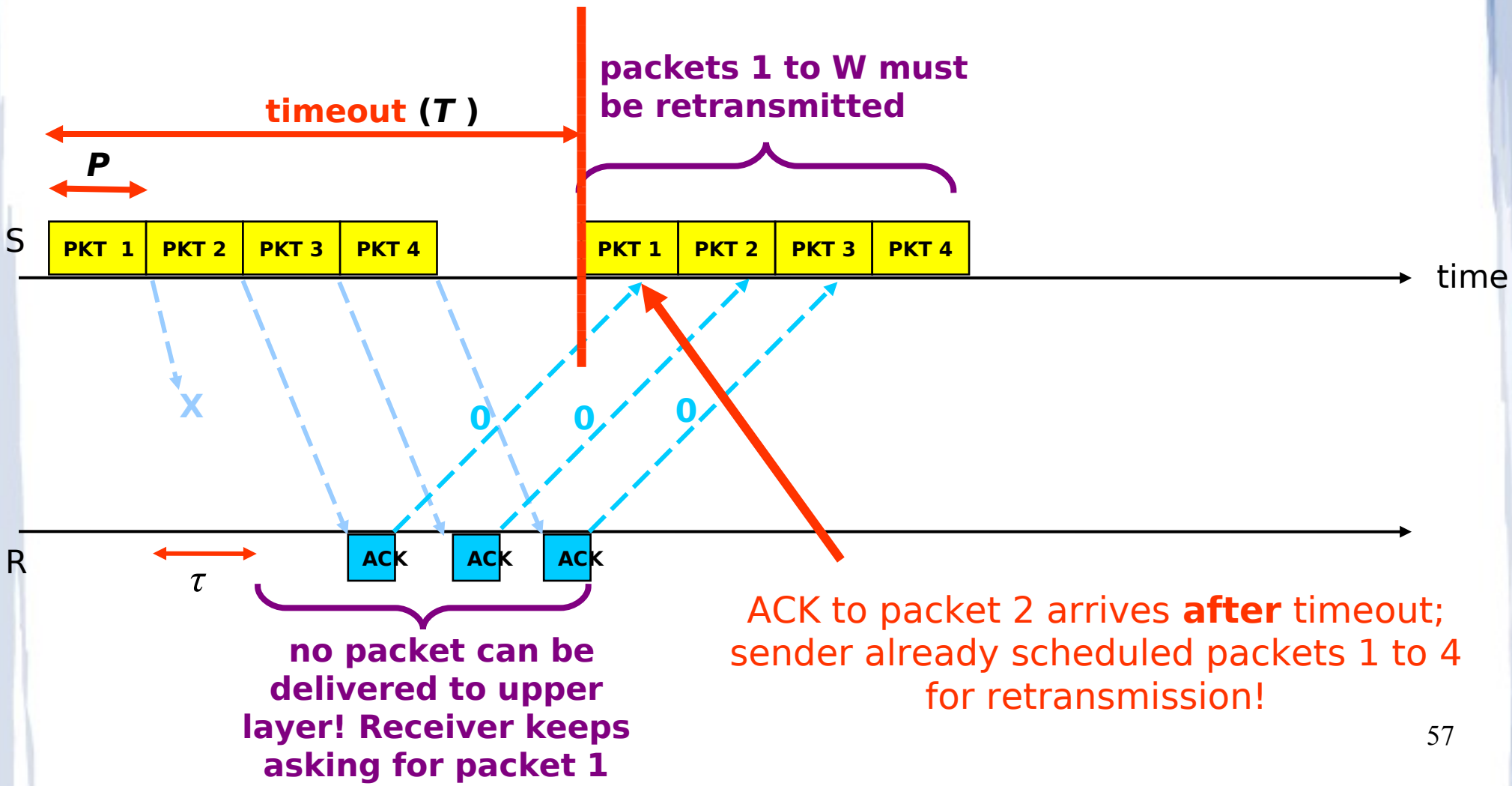
GBN ARQ Exemplo 2



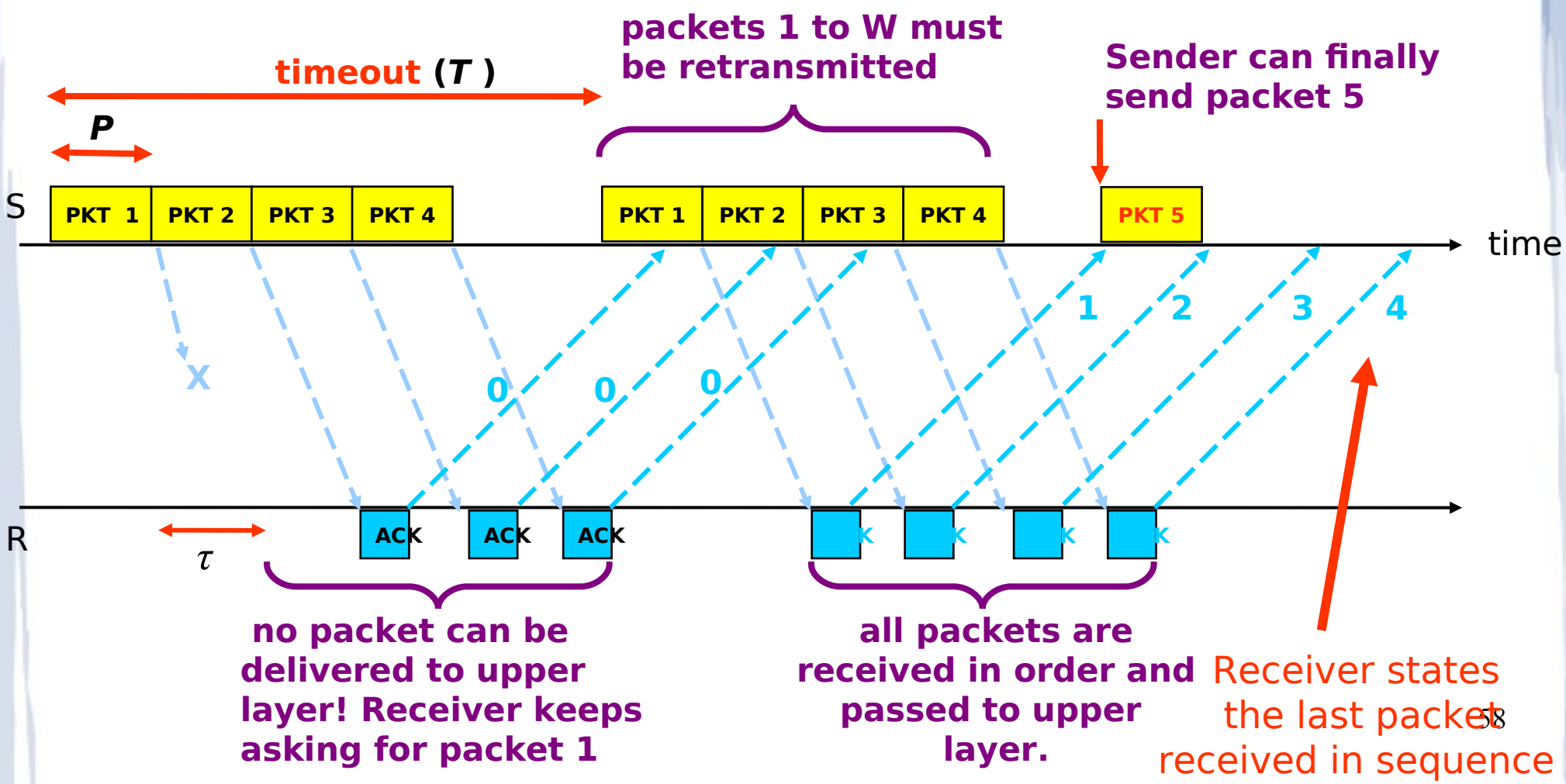
GBN ARQ Exemplo 2



GBN ARQ Exemplo 3

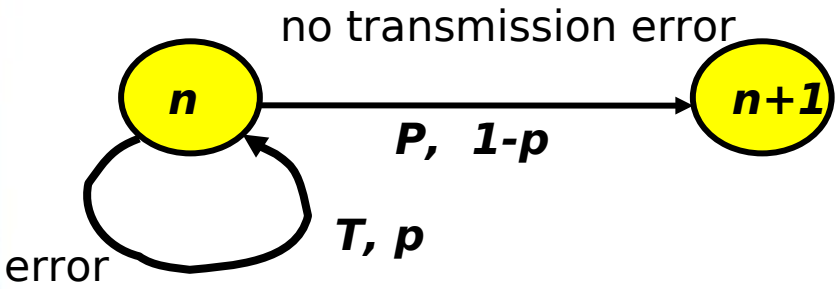


GBN ARQ Exemplo 3



Eficiência do GBN

- Um pacote que não é confirmado desperdiça um *timeout* (*T*)
- Para simplificar a análise, assumiremos ***T= S = W x P***
- Computamos o tempo médio necessário para a fonte obter o ACK de um dado pacote *n* e avançar a janela para *n+1*



$$E\{X\}=(1-p)P+p(T+E\{X\})$$

$$E\{X\}=P+\frac{pT}{1-p}=P\left(1+\frac{pW}{1-p}\right)$$

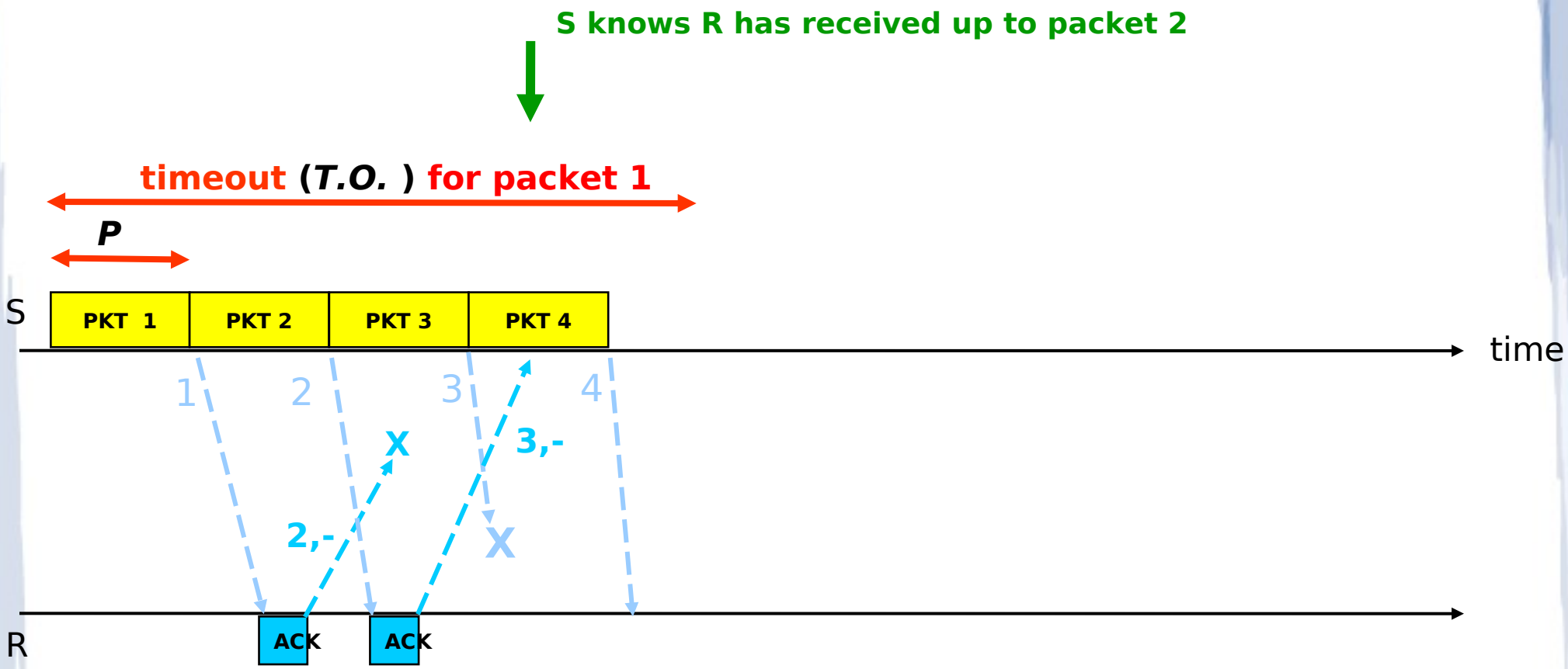
$$Efficiency=\frac{packettime}{E\{X\}}=\frac{P}{E\{X\}}$$

$$\eta_{GBN}=\frac{1}{\left(1+\frac{pW}{1-p}\right)}$$

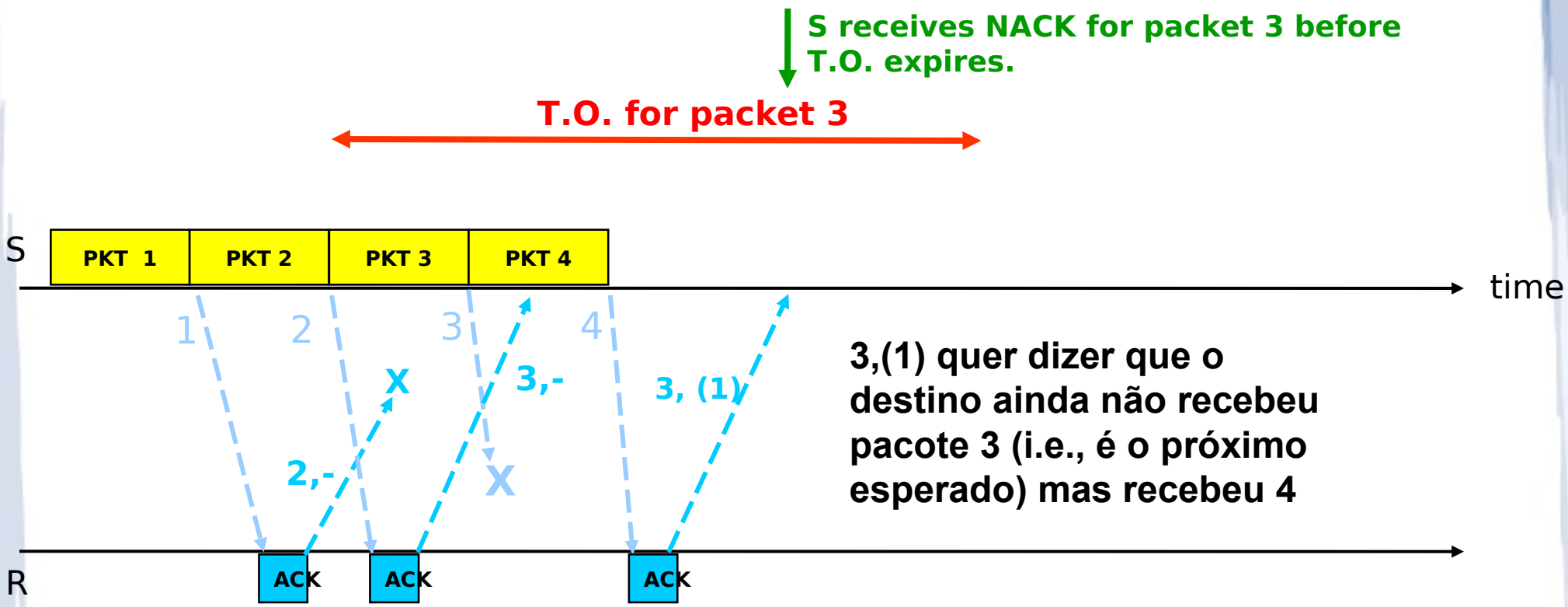
Utilizando **NACKs**

- Até agora assumimos que pacotes são retransmitidos depois de um *timeout*.
- Alternativa: o destino utiliza NACKs para avisar a fonte sobre pacotes em falta possibilitando que a fonte retransmita estes pacotes antes do *timeout*. Isto aumenta a vazão.
- NACKs e ACKs podem ser combinados na janela de ACKs: no exemplo abaixo, o receptor informa que o próximo pacote, em sequência, é o 8; no entanto, ele também informa que não recebeu o 9, recebeu o 10, não recebeu o 11, recebeu o 12, ...

SRP com NACKs

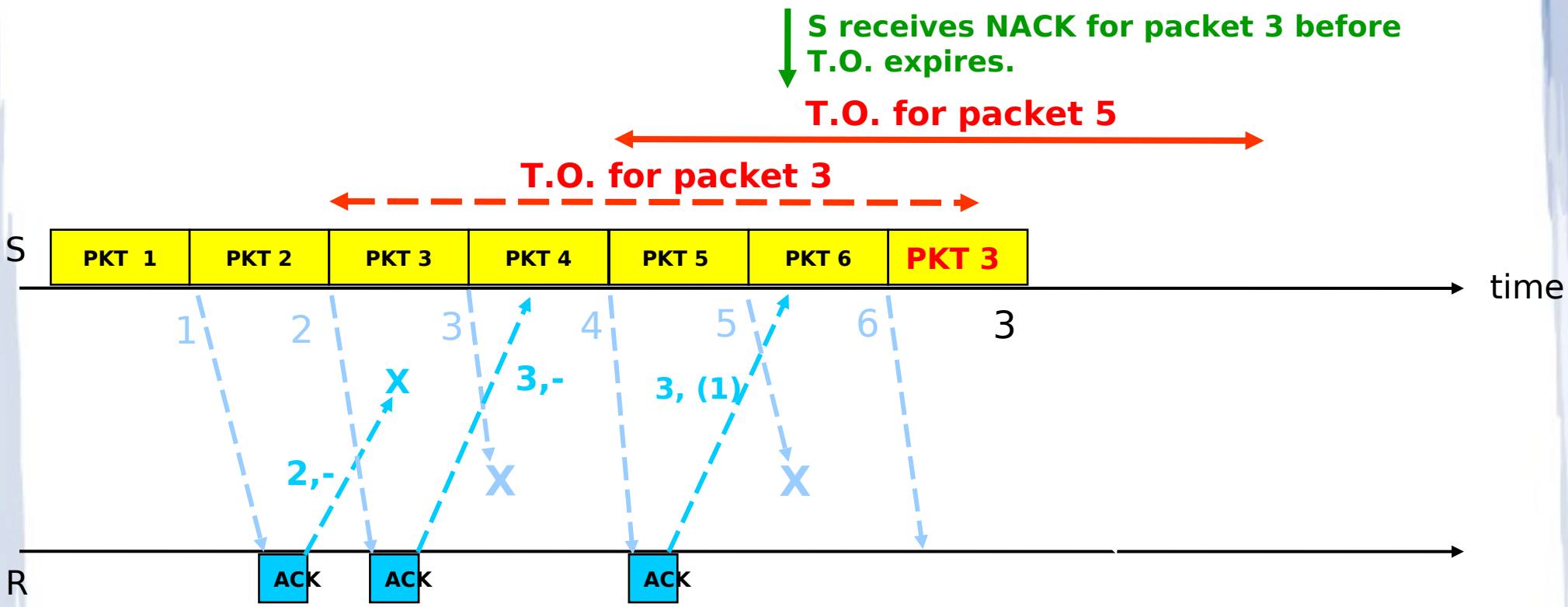


SRP com NACKs

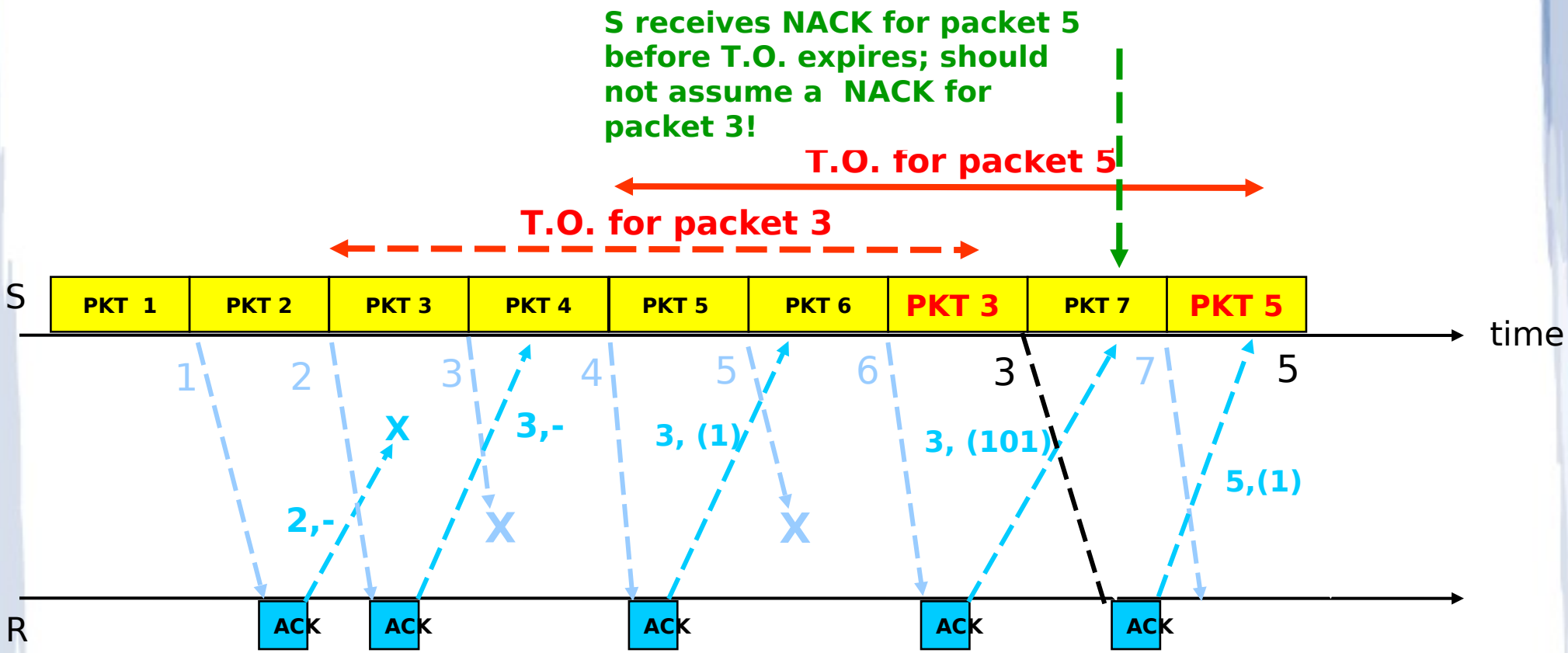


3,(1) quer dizer que o destino ainda não recebeu pacote 3 (i.e., é o próximo esperado) mas recebeu 4

SRP com NACKs

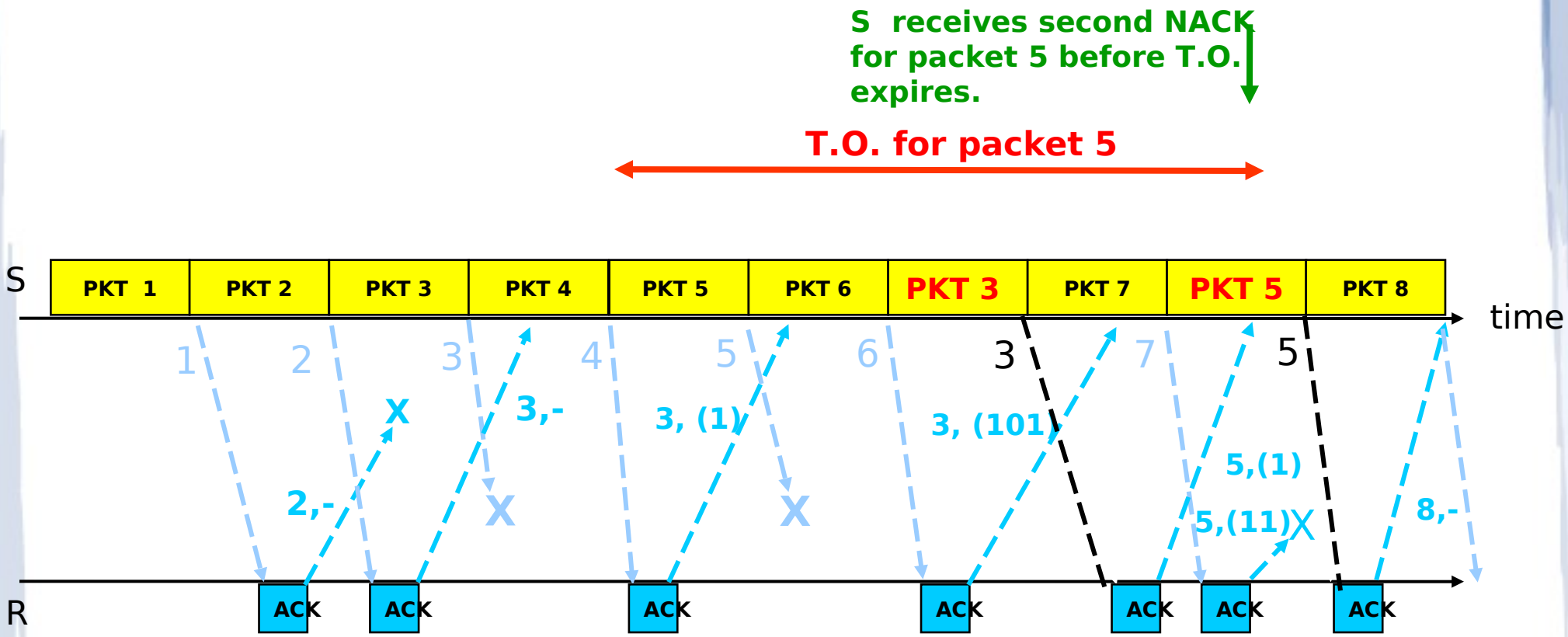


SRP com NACKs



A bit vector set for packet k in the ACK packet should not be interpreted as a NACK to packet k if the ACK packet is received during the next packet time after packet k is (re)transmitted.

SRP com NACKs



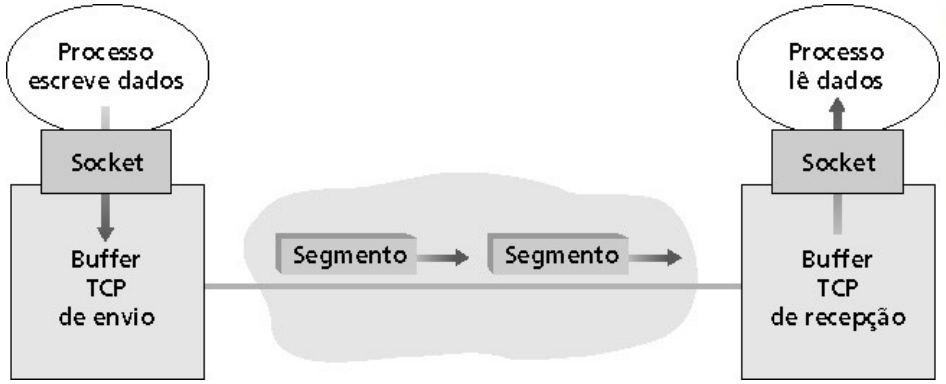
Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não-orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Vestrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

TCP: overview

RFCs: 793, 1122, 1323, 2018, 2581

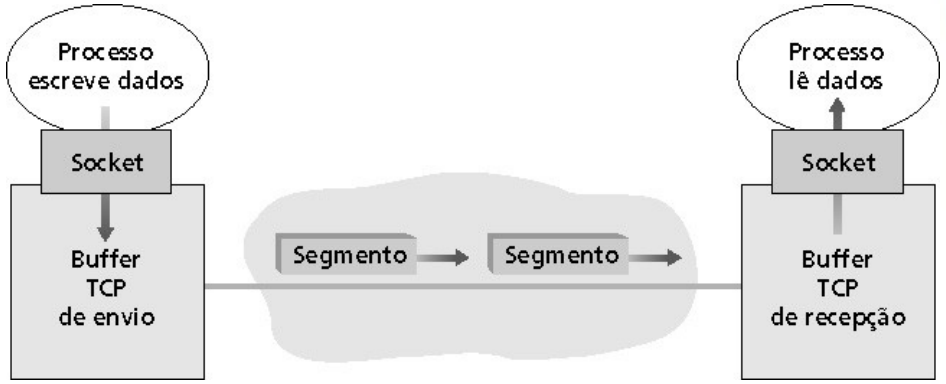
- **Ponto-a-ponto:**
  - Um transmissor, um receptor
- **Confiável, *byte stream* sequencial**
- **Pipelined:** (transmissão de vários pacotes sem necessidade de confirmação individual)
  - Controle de congestão e de fluxo definem tamanho da janela



TCP: overview

RFCs: 793, 1122, 1323, 2018, 2581

- **Buffers de transmissão e de recepção**
- Dados *full-duplex*:
  - Transmissão bidirecional na mesma conexão
  - MSS: *maximum segment size*
- Orientado à conexão:
  - Apresentação (troca de mensagens de controle) inicia o estado do transmissor e do receptor antes da troca de dados
- Controle de fluxo:
  - Transmissor não esgota (afoga) a capacidade do receptor



Estrutura do segmento TCP

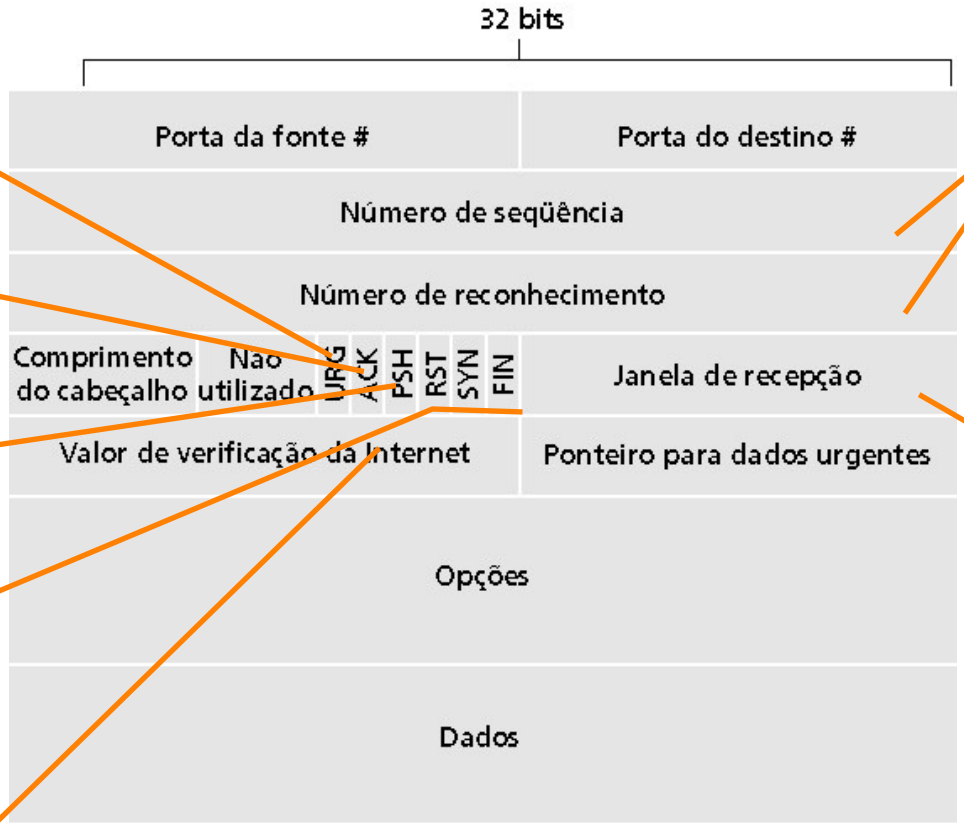
URG: dados urgentes (pouco usados)

ACK: campo de ACK é válido

PSH: produz envio de dados (pouco usado)

RST, SYN, FIN: estabelec. de conexão (comandos de criação e término)

Internet checksum (como no UDP)



contagem por bytes de dados (não segmentos!)

número de bytes receptor está pronto para aceitar

Número de seqüência e ACKs do TCP

Números de seqüência:

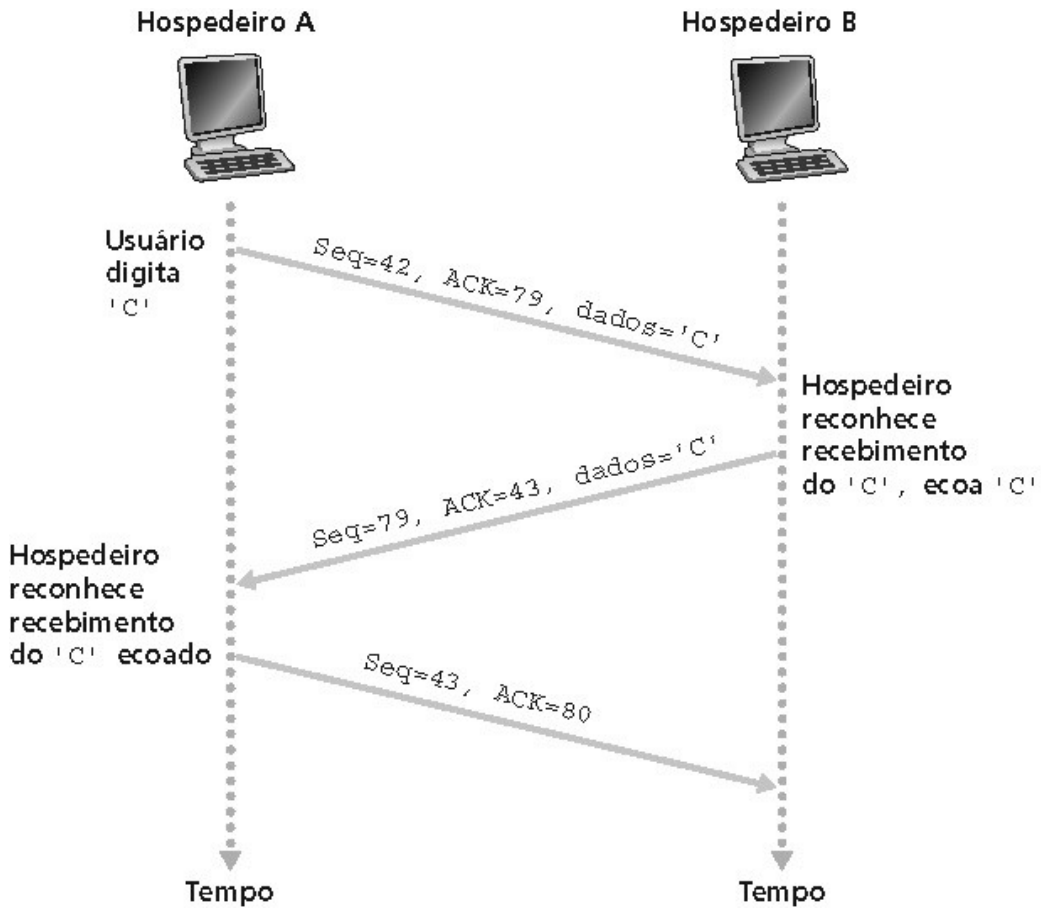
- Número do primeiro *byte* no segmento de dados enviado

ACKs:

- Número do próximo *byte* esperado do outro lado
- ACK acumulativo

P.: Como o receptor trata segmentos fora de ordem?

- A especificação do TCP não define, fica a critério de quem implementa!!!

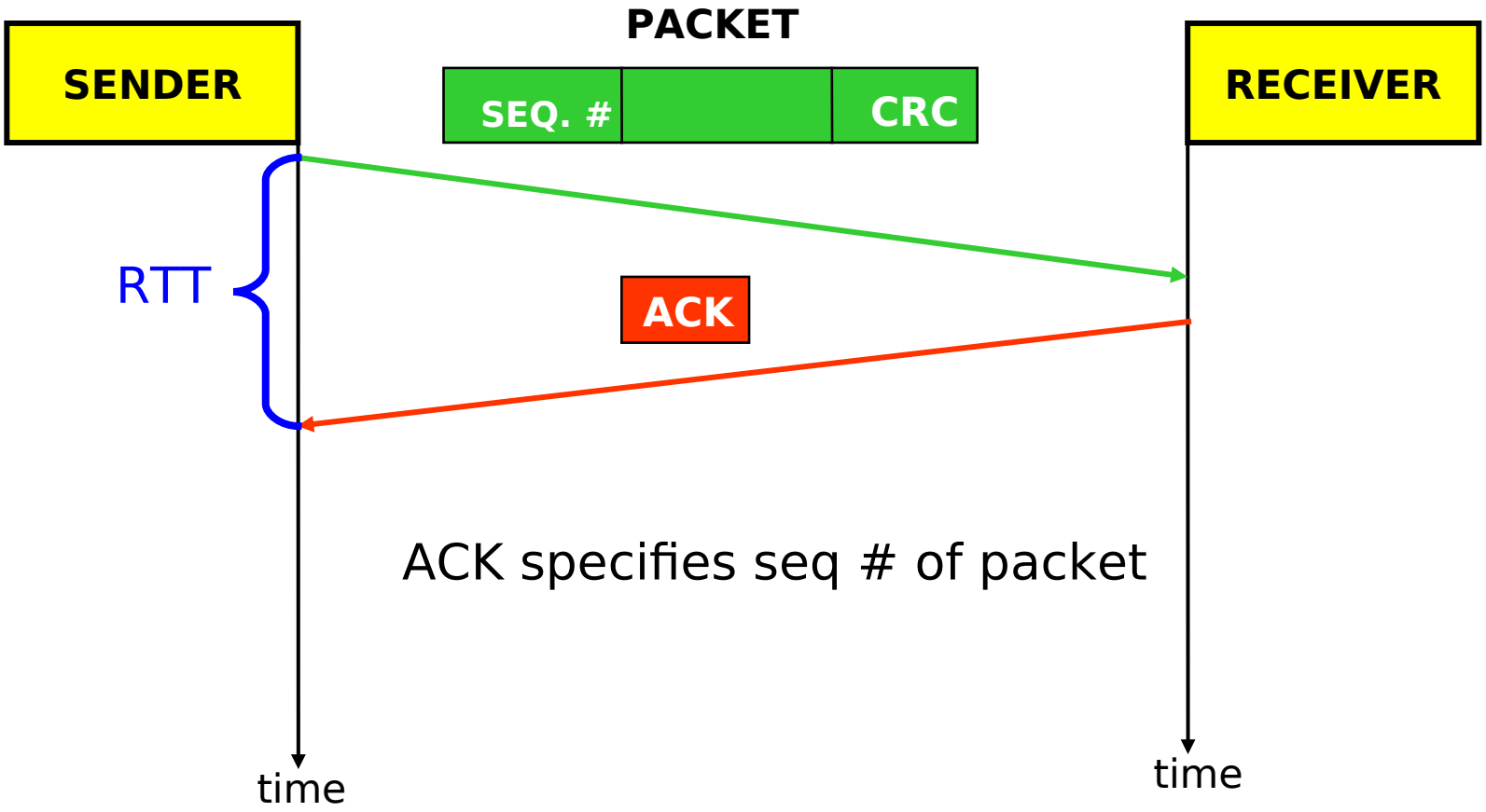


Temporização no TCP

- Temporização não pode ser um valor fixo e predefinido
- Depende da “distância” entre o par de nós comunicantes: deve ser baseado no *round trip time* (RTT)

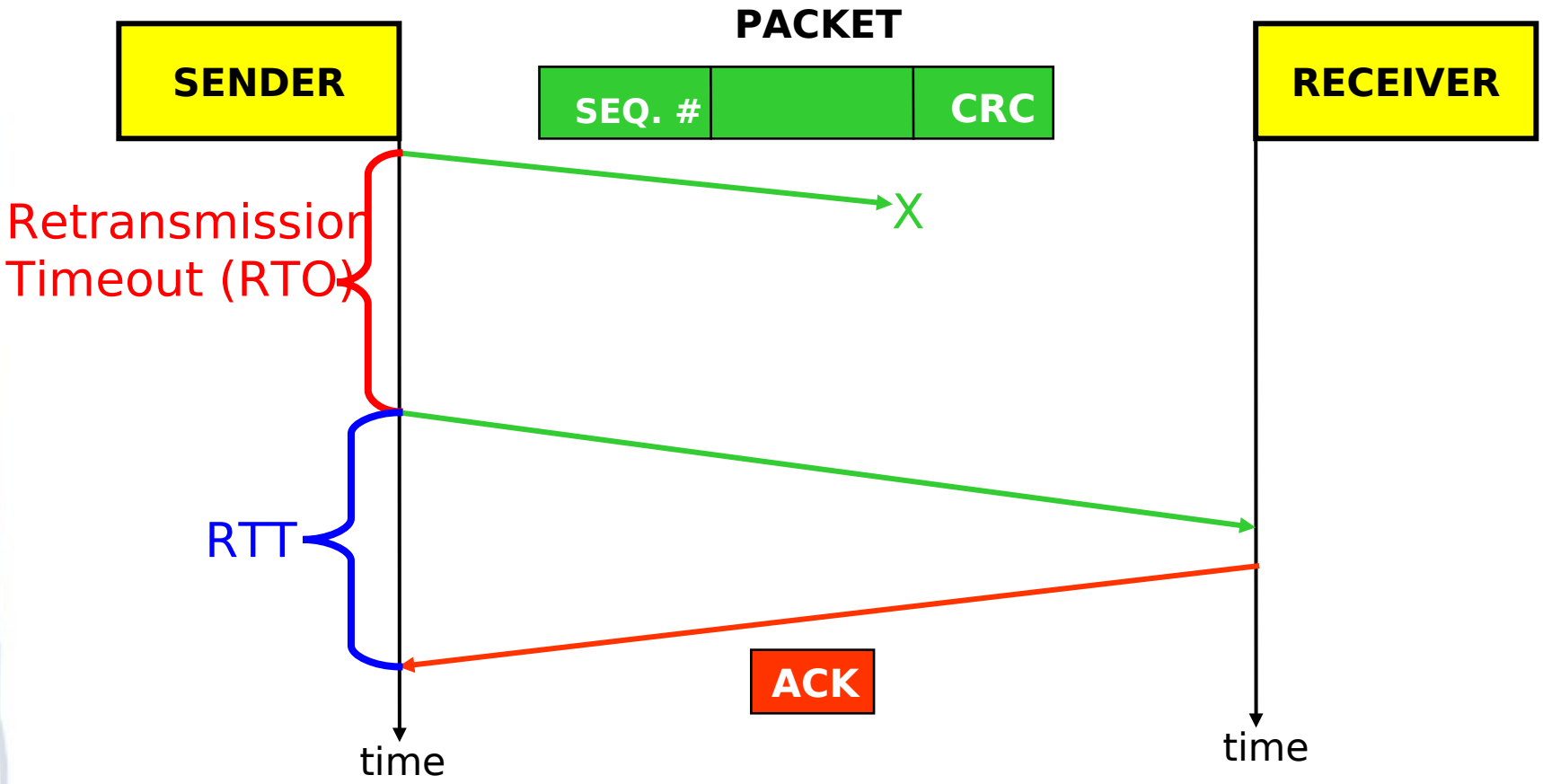
Estimativa de RTT em enlaces não FIFO

Assumindo que somente o # sequência é incluído no cabeçalho!!!



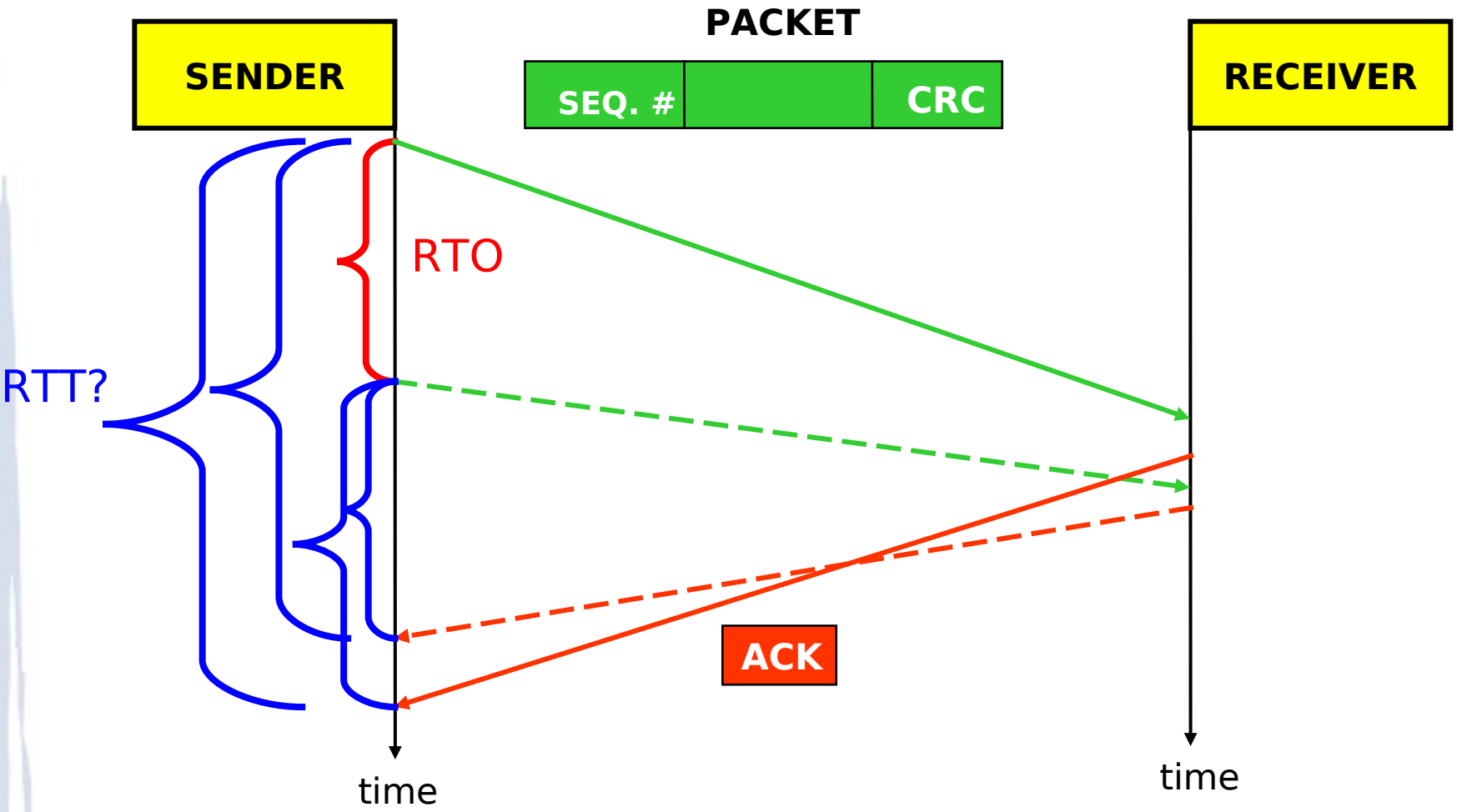
Estimativa de RTT em enlaces não FIFO

Assumindo que somente o # sequência é incluído no cabeçalho!!!



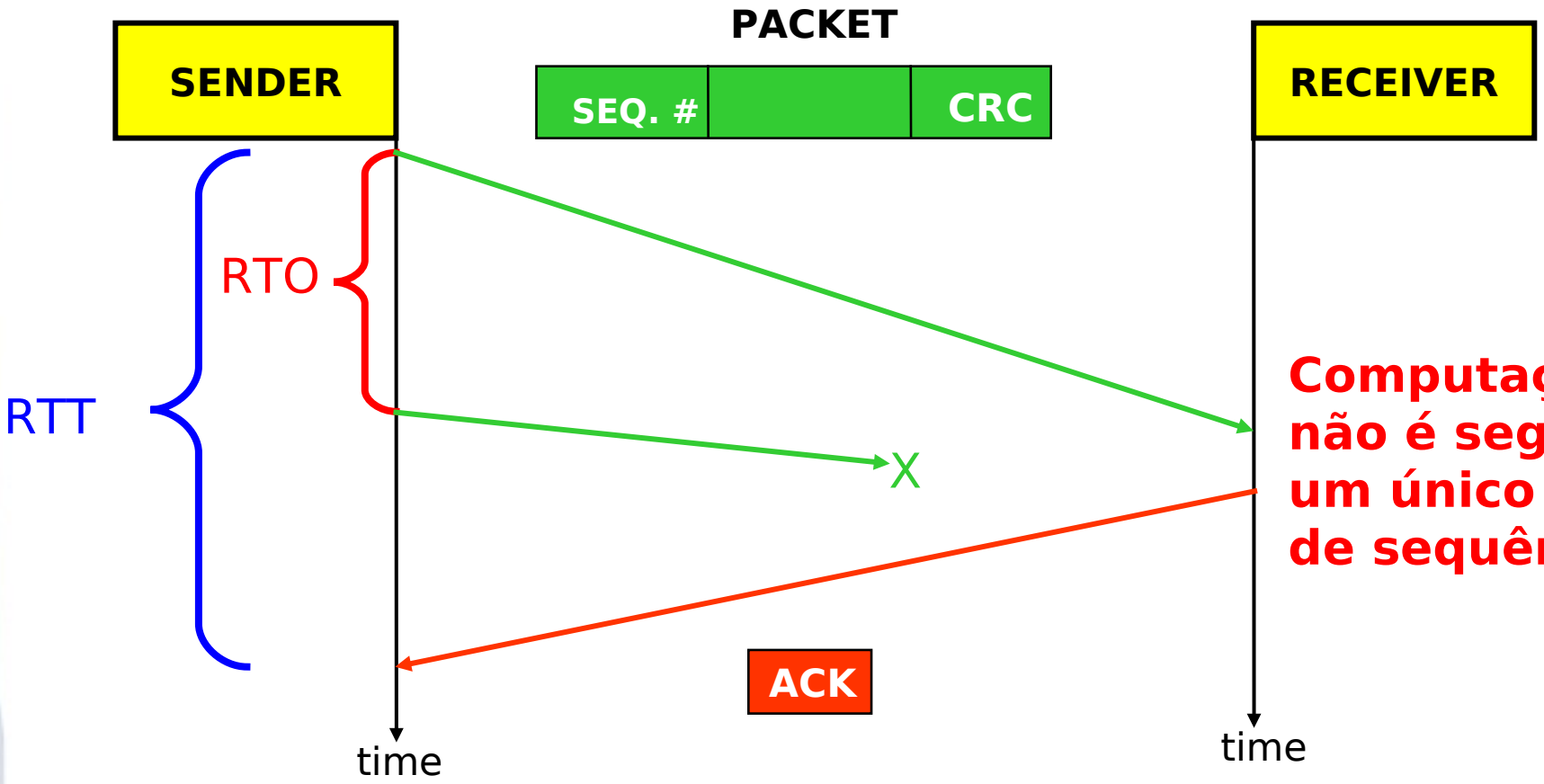
Estimativa de RTT em enlaces não FIFO

Assumindo que somente o # sequência é incluído no cabeçalho!!!



Estimativa de RTT em enlaces não FIFO

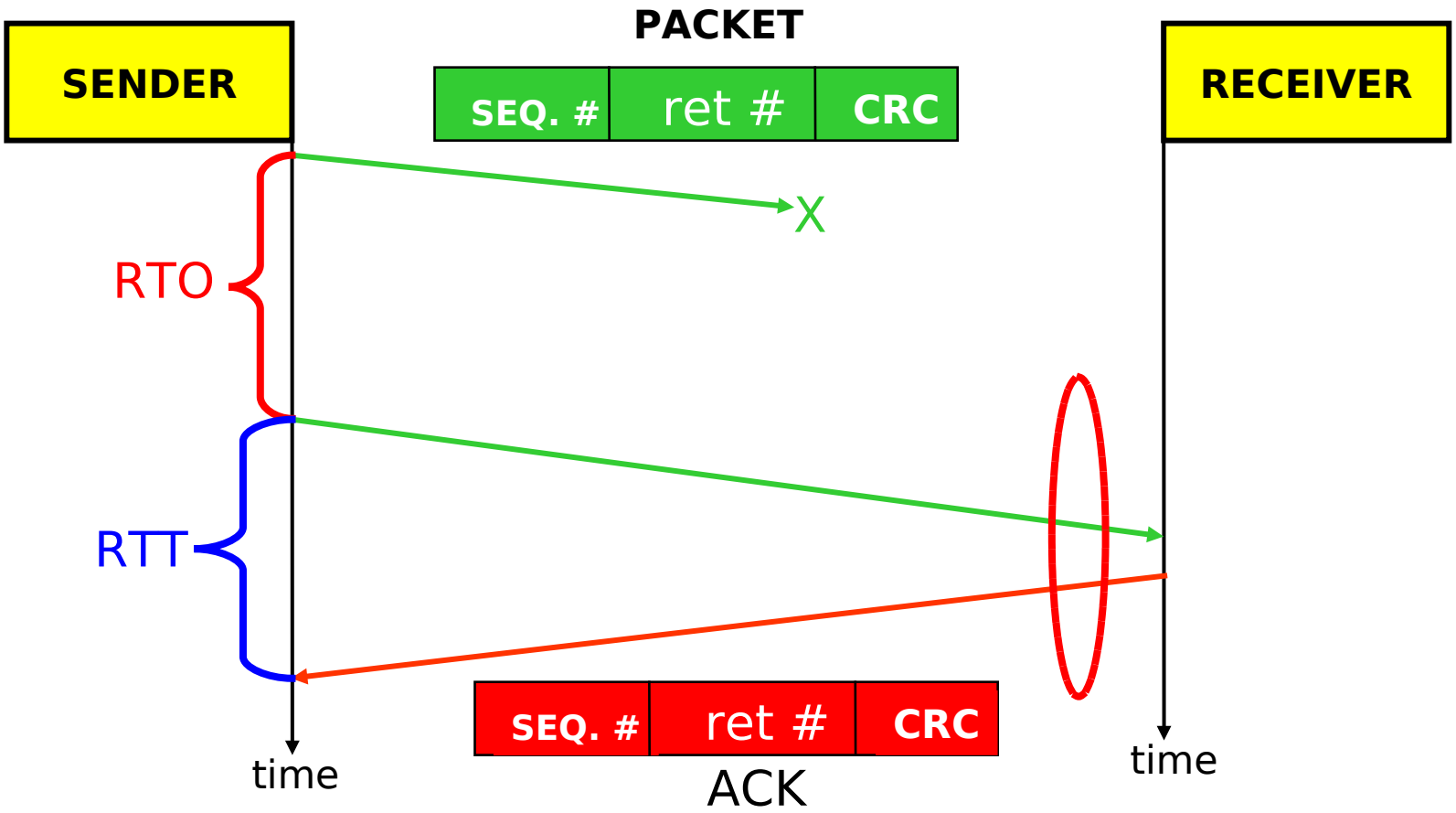
Assumindo que somente o # sequência é incluído no cabeçalho!!!



Computação do RT não é segura com um único número de sequência.

Estimativa de RTT em enlaces não FIFO

ACK deve ser atrelado a uma transmissão específica, devendo-se incluir, além do # de sequência, o # da Retransmissão.



TCP: *Round Trip Time* e temporização (*timeout*)

P.: como escolher o valor da temporização (*timeout*) do TCP?

- Maior que o RTT
  - Mas RTT varia
    - Muito curto: temporização prematura
      - Retransmissões desnecessárias
    - Muito longo: a reação à perda de segmento fica lenta

P.: Como estimar o RTT?

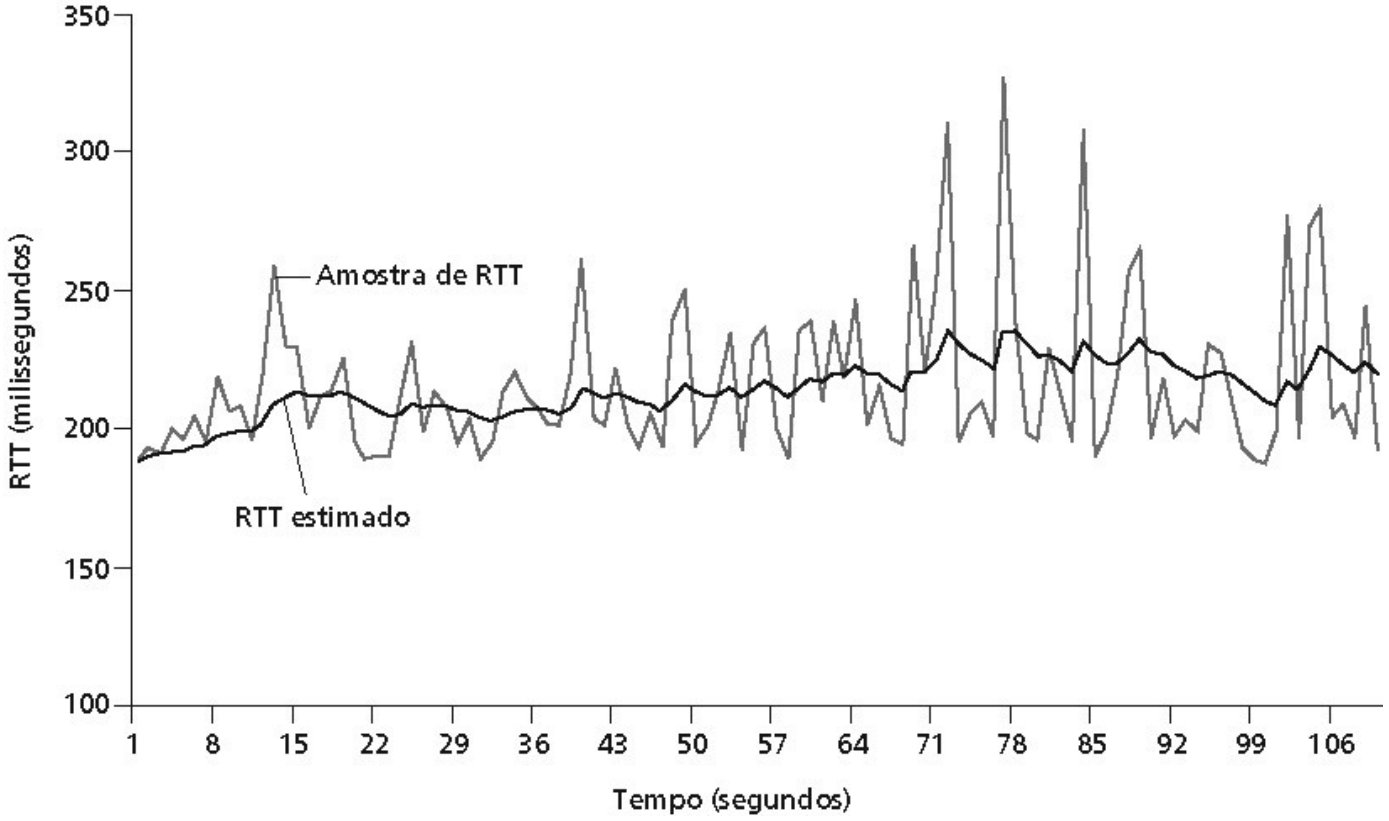
- SampleRTT: tempo medido da transmissão de um segmento até a Respectiva confirmação
  - Ignora retransmissões e segmentos reconhecidos de forma cumulativa
- **SampleRTT** varia de forma rápida, é desejável um amortecedor para a estimativa do RTT
  - Usar várias medidas recentes, não apenas o último **SampleRTT** obtido

TCP: *Round Trip Time* e temporização (*timeout*)

$$\texttt{EstimatedRTT} = (1-\alpha)*\texttt{EstimatedRTT} + \alpha*\texttt{SampleRTT}$$

- Média móvel com peso exponencial
- Influência de uma dada amostra decresce de forma exponencial
- Valor típico: α = 0,125

Exemplos de estimativa do RTT



TCP: *Round Trip Time* e temporização (*timeout*)

Definindo a temporização

- **`EstimatedRTT`** mais "margem de segurança"
  - Grandes variações no **`EstimatedRTT`** **->** maior margem de segurança
- Primeiro estimar o quanto o SampleRTT se desvia do EstimatedRTT:

```
DevRTT = (1-β)*DevRTT +
             β*|SampleRTT-EstimatedRTT|

(typically, β = 0.25)
```

Então ajustar o intervalo de temporização

`TimeoutInterval = EstimatedRTT + 4*DevRTT`

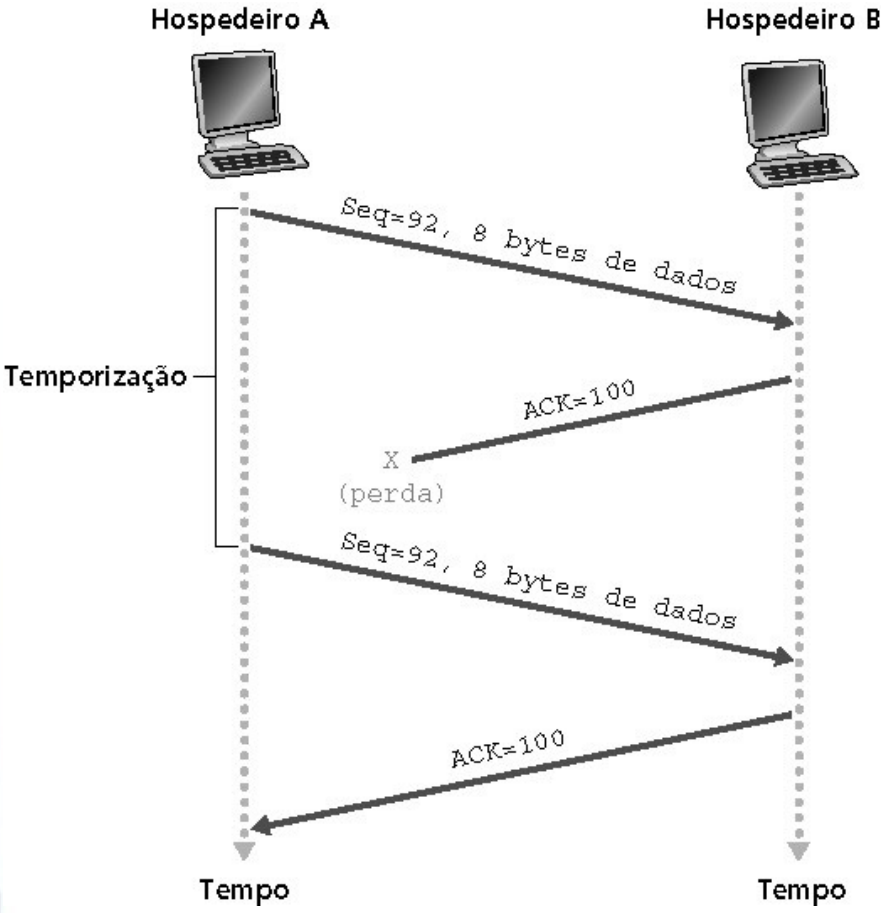
Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não-orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

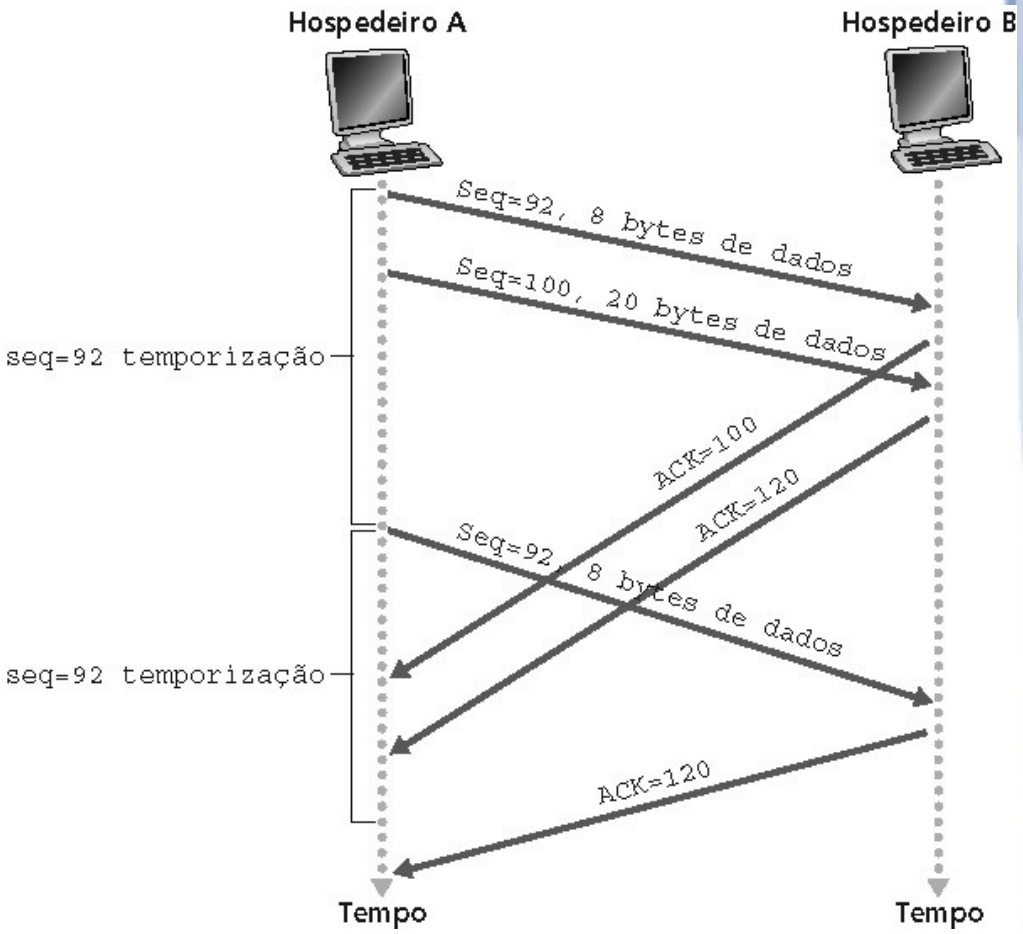
TCP: transferência de dados confiável

- TCP cria serviços de *reliable data transfer* (rdt) em cima do serviço não-confiável do IP (*best effort*)
- *Pipelined segments*
- ACKs cumulativos
- TCP usa tempo de retransmissão simples
- Retransmissões são disparadas por:
    - Eventos de tempo de confirmação (i.e., *timeout*)
    - ACKs duplicados

TCP: cenários de retransmissão

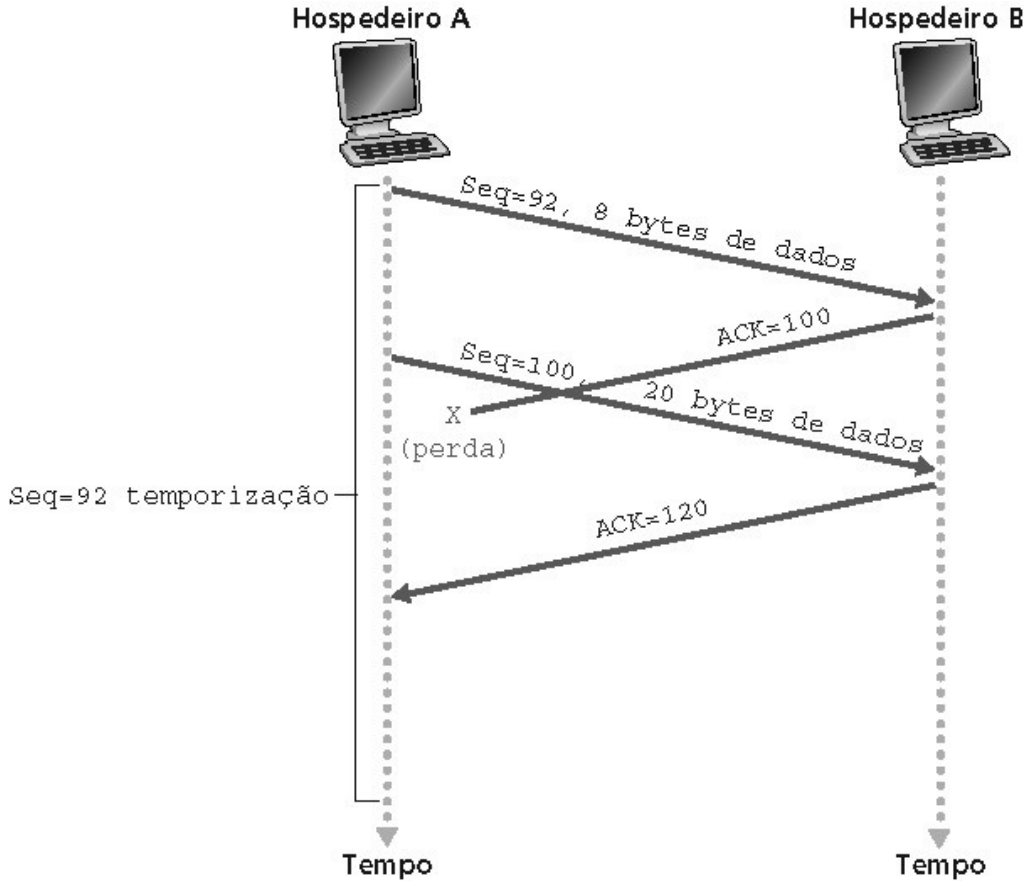


Cenário com perda do ACK



Temporização prematura, ACKs cumulativos

TCP: cenários de retransmissão



Cenário de **ACK cumulativo**

Geração de ACK [RFC 1122, RFC 2581]

Evento no receptor	Ação do receptor TCP
Segmento chega em ordem, não há lacunas, segmentos anteriores já aceitos	ACK retardado. Espera até 500 ms pelo próximo segmento. Se não chegar, envia ACK
Segmento chega em ordem, não há lacunas, um ACK atrasado pendente	Imediatamente envia um ACK cumulativo
Segmento chega fora de ordem, número de seqüência chegou maior: *gap* detectado	Envia ACK duplicado, indicando número de seqüência do próximo *byte* esperado
Chegada de segmento que parcial ou completamente preenche o *gap*	Reconhece imediatamente se o segmento começa na borda inferior do *gap*

Retransmissão rápida

- Com freqüência, o tempo de expiração é relativamente longo:
  - Longo atraso antes de reenviar um pacote perdido
- Detecta segmentos perdidos por meio de ACKs duplicados
  - Transmissor freqüentemente envia muitos segmentos *back-to-back*
  - Se o segmento é perdido, haverá muitos ACKs duplicados.
- Se o transmissor recebe **3 ACKs** para o mesmo dado, ele supõe que o segmento com confirmações duplicadas foi perdido:
  - Retransmissão rápida: reenvia o segmento antes de o temporizador expirar

Algoritmo de retransmissão rápida

```
event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase = y
                if (there are currently not-yet-acknowledged segments)
                    start timer
              }
            else {
                increment count of dup ACKs received for y
                if (count of dup ACKs received for y = 3) {
                    resend segment with sequence number y
                }
```

ACK duplicado para um segmento já confirmado

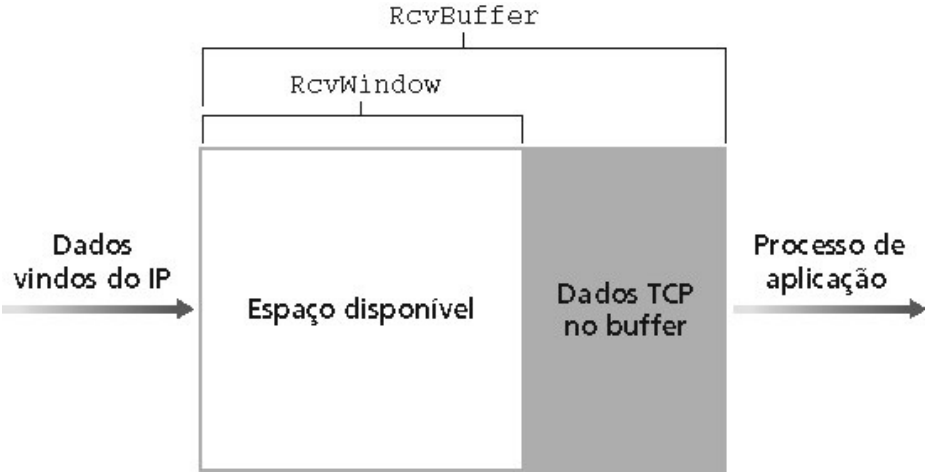
retransmissão rápida

Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

TCP: controle de fluxo

- lado receptor da conexão TCP possui um *buffer* de recepção:



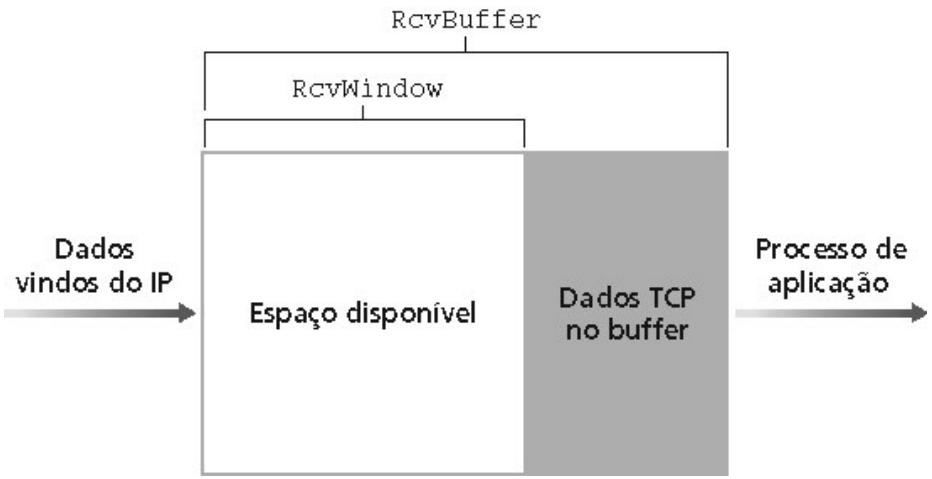
- Processos de aplicação podem ser lentos para ler o *buffer*

Controle de fluxo

Transmissor não deve esgotar os *buffers* de recepção enviando dados rápido demais

- Serviço de ***speed-matching***: encontra a taxa de envio adequada à taxa de vazão da aplicação receptora

Controle de fluxo TCP: como funciona



- Receptor informa a área disponível incluindo valor **`RcvWindow`** nos segmentos
- Transmissor limita os dados não confimados ao **`RcvWindow`**
  - Garantia contra overflow no buffer do receptor

(suponha que o receptor TCP descarte segmentos fora de ordem)

- Espaço disponível no buffer

= `RcvWindow`

= `RcvBuffer-[LastByteRcvd - LastByteRead]`

Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Gerenciamento de conexão TCP

TCP transmissor estabelece conexão com o receptor antes de trocar segmentos de dados

- Inicializar variáveis:
  - Números de seqüência
  - *Buffers*, controle de fluxo (ex. **`RcvWindow`**)
- **Cliente**: inicia a conexão

```
Socket clientSocket = new   Socket("hostname","port number");
```

- **Servidor**: chamado pelo cliente

```
Socket connectionSocket = welcomeSocket.accept();
```

Three way handshake:

Passo 1: sistema final cliente envia TCP SYN ao servidor

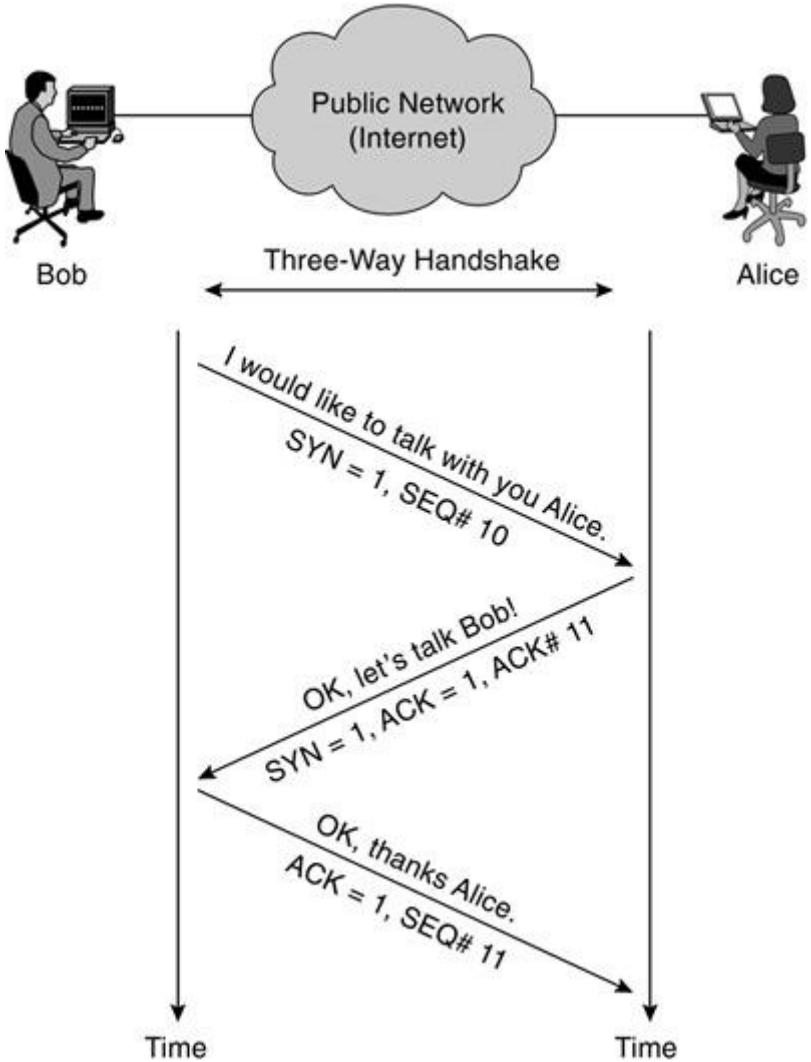
- Especifica número de seqüência inicial

Passo 2: sistema final servidor que recebe o SYN, responde com segmento SYN/ACK

- Reconhece o SYN recebido (i.e., responde com SYN/ACK)
- Aloca *buffers*
- Especifica o número de seqüência inicial do servidor

Passo 3: o sistema final cliente reconhece o SYN/ACK

Gerenciamento de conexão TCP: *three-way handshake*



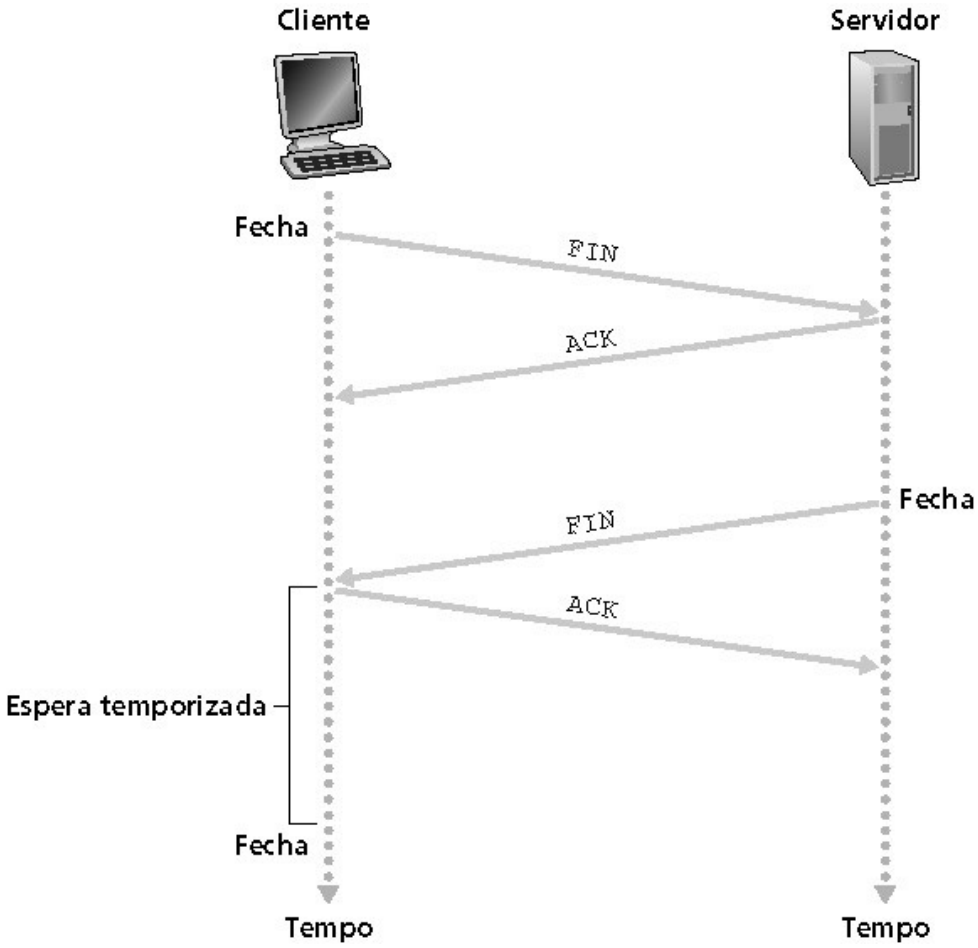
Gerenciamento de conexão TCP

Fechando uma conexão:

cliente fecha o socket:
clientSocket.close();

Passo 1: o cliente envia o segmento TCP FIN ao servidor

Passo 2: servidor recebe FIN, responde com ACK. Fecha a conexão, envia FIN



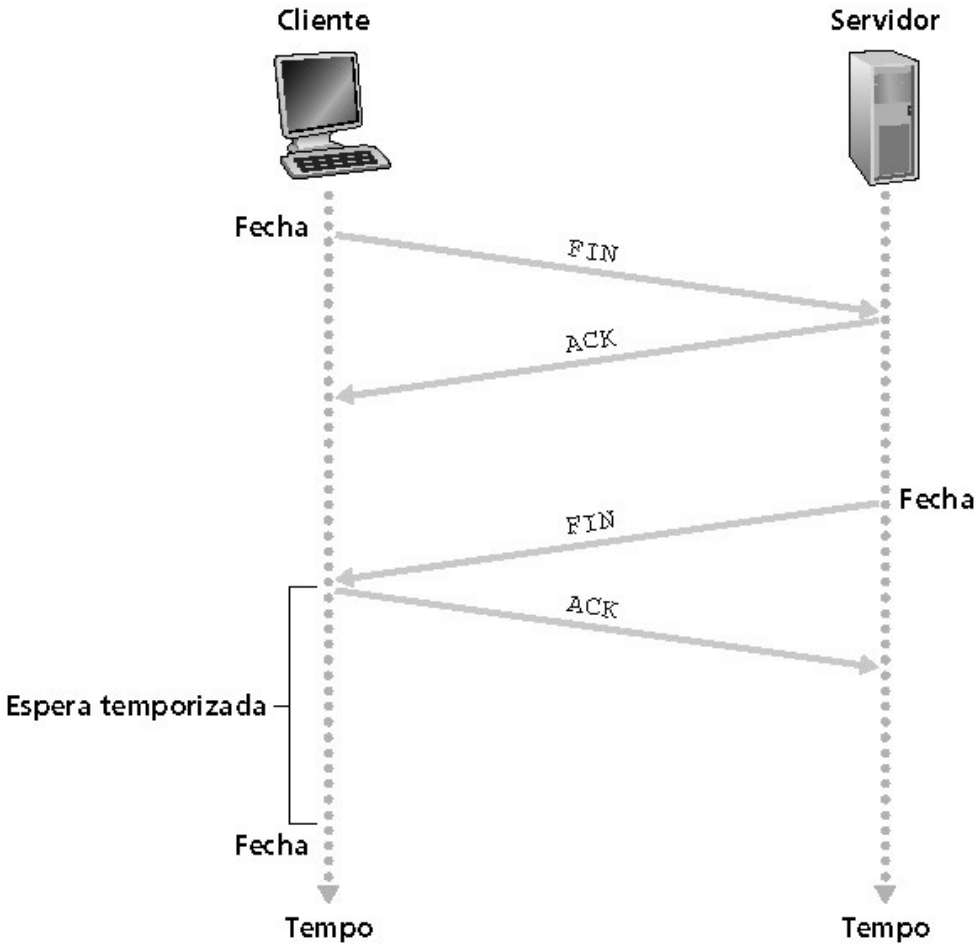
Gerenciamento de conexão TCP

Passo 3: cliente recebe FIN, responde com ACK.

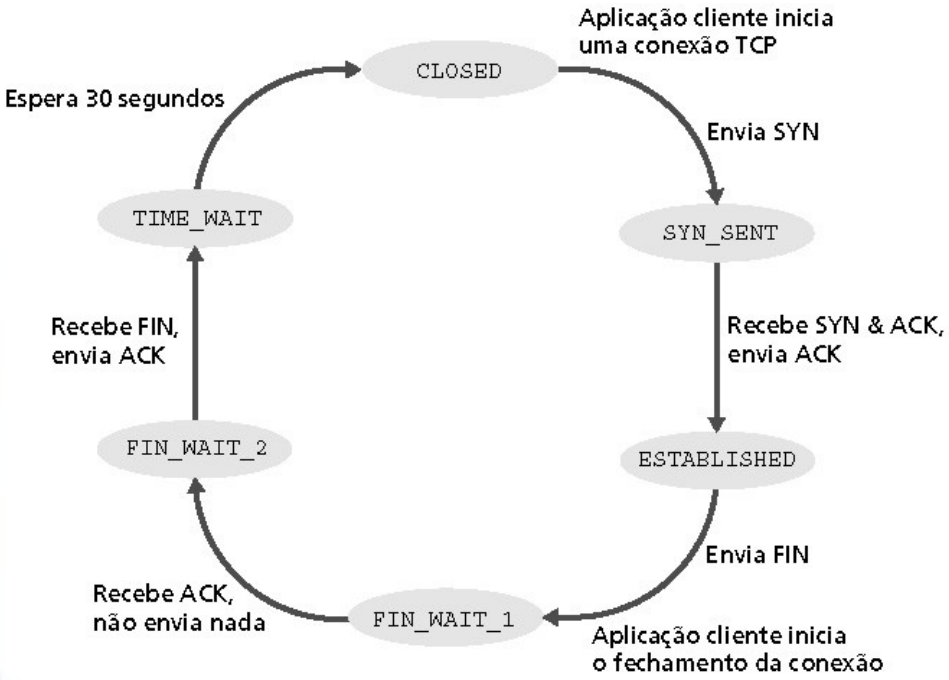
- Entra "espera temporizada" - vai responder com ACK a FINs recebidos

Passo 4: servidor, recebe ACK. Conexão fechada

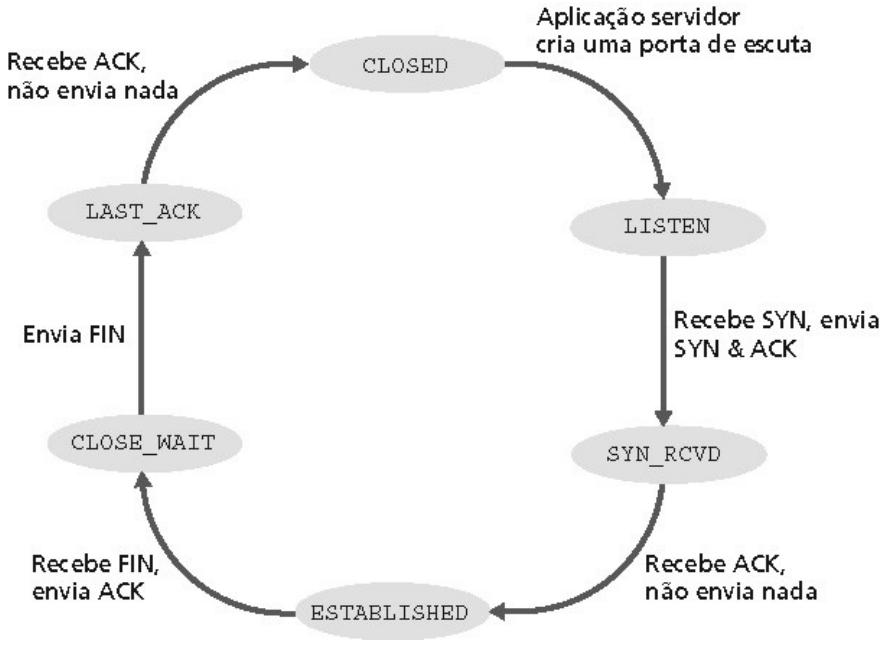
OBS.: tempo de espera essencial para tratar (i.e., descartar) possíveis pacotes pertencentes à conexão ainda em trânsito na rede!



Gerenciamento de conexão TCP



Estados do cliente



Estados do servidor

Camada de transporte

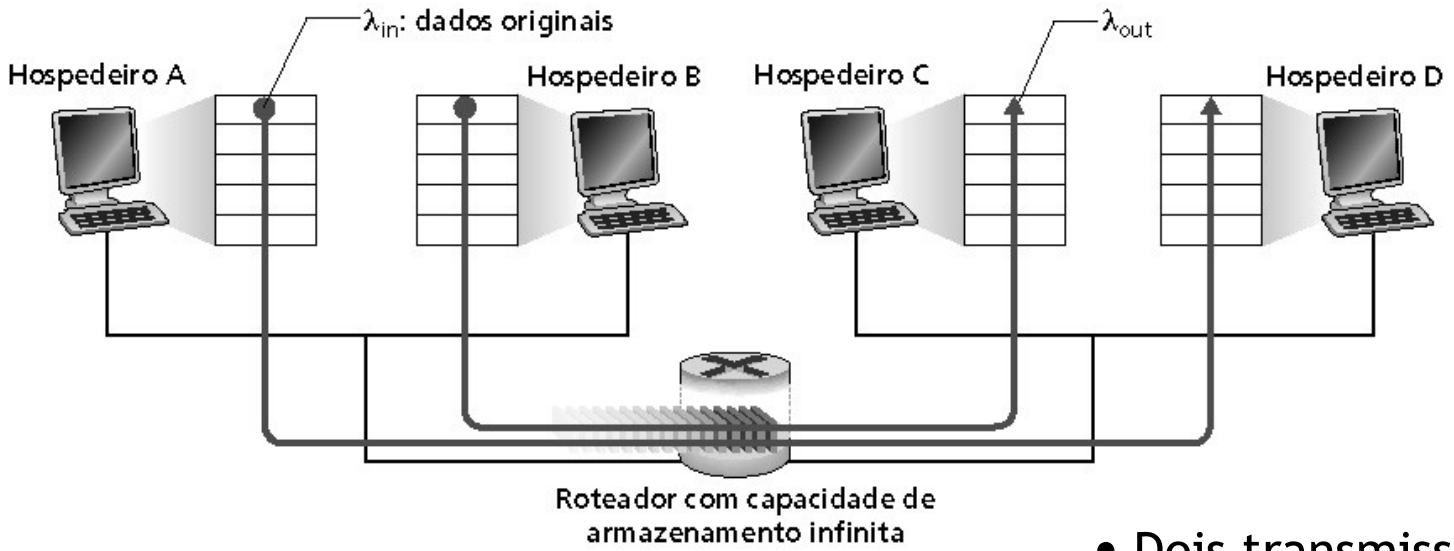
- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

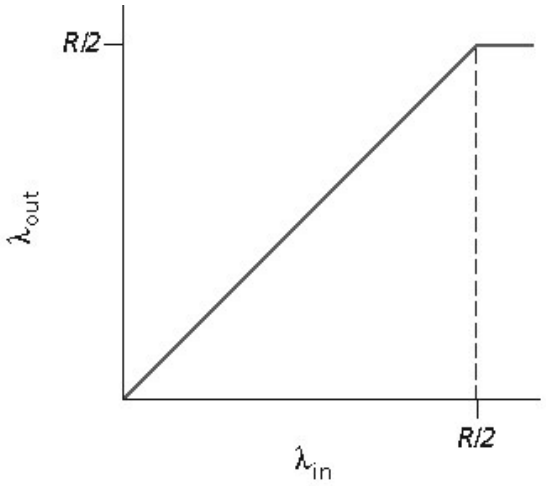
Princípios de controle de congestionamento

Congestionamento:

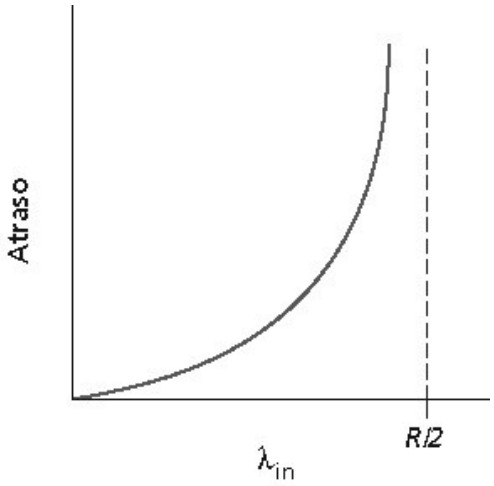
- Informalmente: "muitas fontes enviando dados acima da capacidade da **rede** de tratá-los"
- Diferente de controle de fluxo:
  - Envolve a rede e não o destino final!!
- **Sintomas:**
  - Perda de pacotes (saturação de *buffer* nos roteadores)
  - Atrasos crescentes (filas nos *buffers* dos roteadores)

Causas/custos do congestionamento: cenário 1





a.



b.

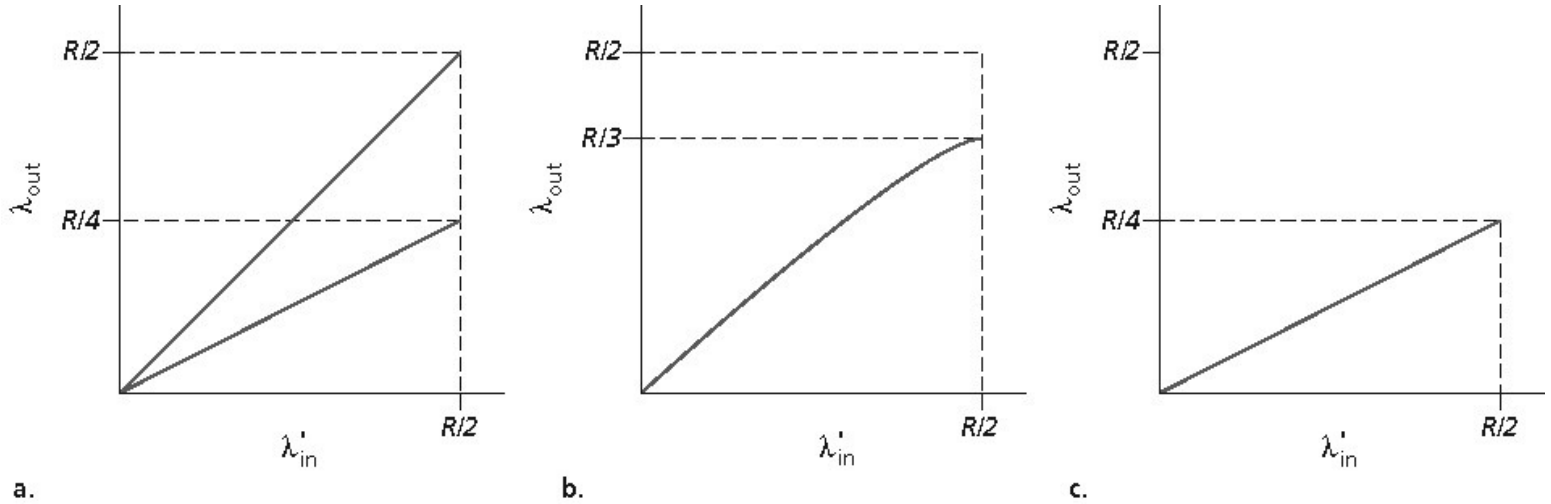
- Dois transmissores, dois receptores
- Um roteador, ***buffers* infinitos**
- Não há retransmissão
- Grandes atrasos quando congestionado
- Máxima vazão alcançável

Causas/custos do congestionamento: cenário 2

- Um roteador com *buffers* finitos
- Transmissor reenvia pacotes perdidos

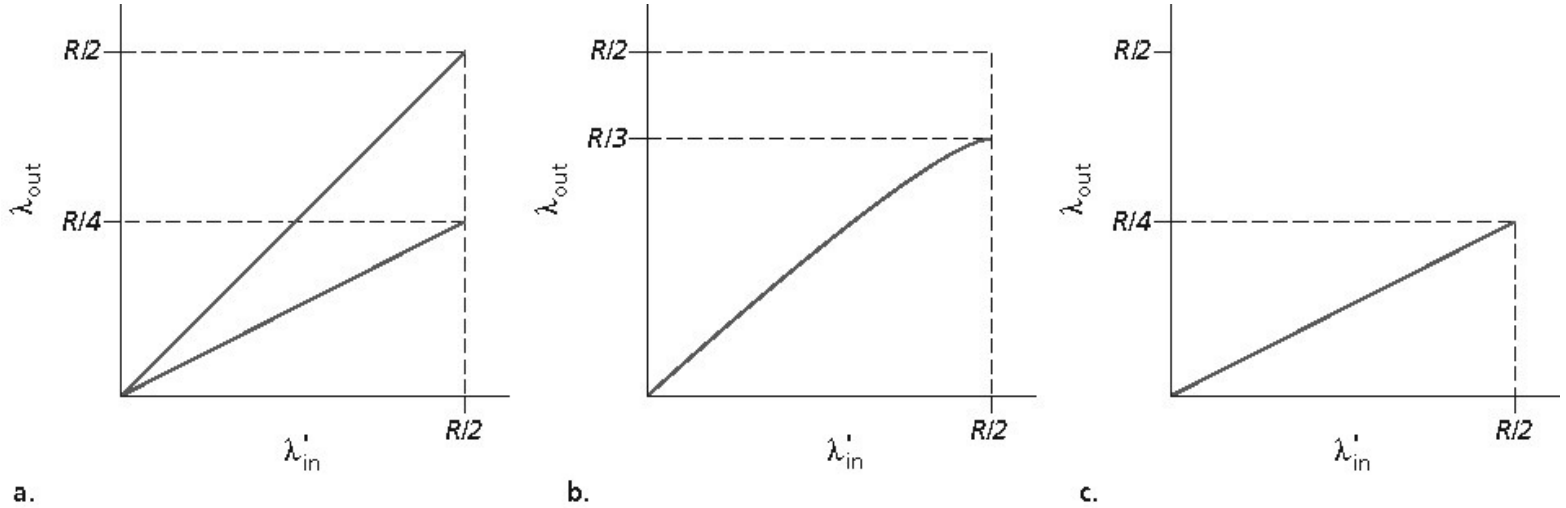
Causas/custos do congestionamento: cenário 2

- Hipótese: a fonte só envia um pacote quando existe espaço no *buffer* do roteador para armazenar o pacote (a fonte simplesmente sabe disso, como que por um passe de mágica! :)
- Neste caso, naturalmente, não ocorre retransmissões
- No exemplo ilustrado na **sub-figura (a)** abaixo, a curva superior mostra que o fluxo de saída é igual ao fluxo de envio, uma vez que não ocorrem retransmissões!



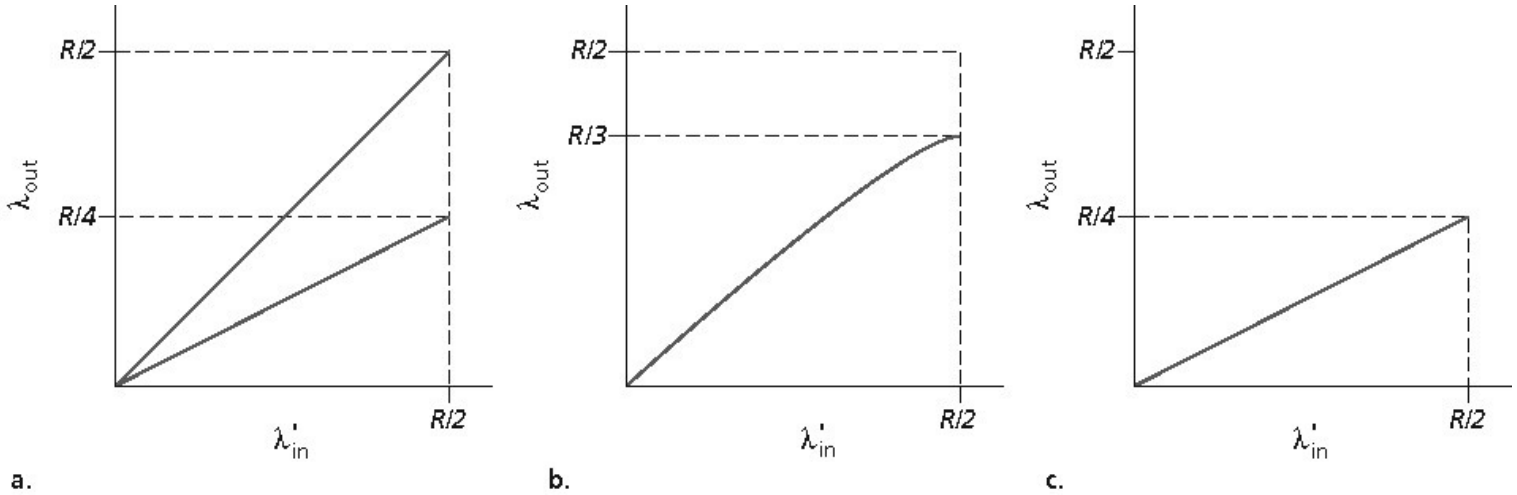
Causas/custos do congestionamento: cenário 2

- Hipótese: a fonte somente retransmite pacotes realmente perdidos (isso seria possível de detectar assumindo que o *timeout* seja calculado apropriadamente)
- No exemplo ilustrado na **sub-figura (b)**, o fluxo máximo de entrega de dados ao destino corresponde a R/3: ou seja 0.33333R são dados novos e 0.1666R são dados repetidos!



Causas/custos do congestionamento: cenário 2

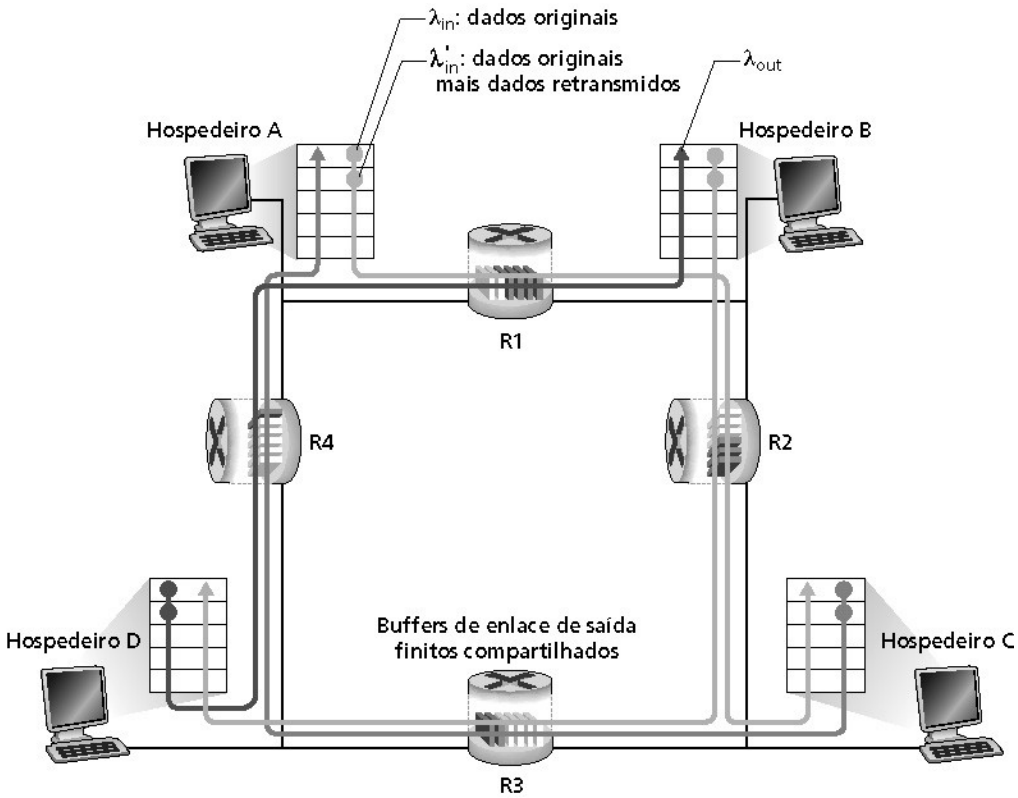
- Hipótese: a fonte estima um RTO (i.e., *timeout* de retransmissão) muito curto. Neste caso, aumenta a probabilidade da fonte retransmitir pacotes desnecessariamente.
- No exemplo da **sub-figura(c)** cada pacote é transmitido, em média, duas vezes (original + cópia), fazendo com que a vazão caia à metade (i.e., 0.25R).



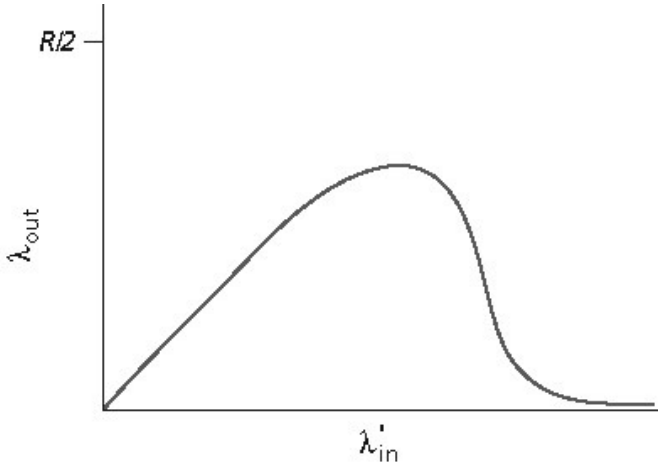
Causas/custos do congestionamento: cenário 3

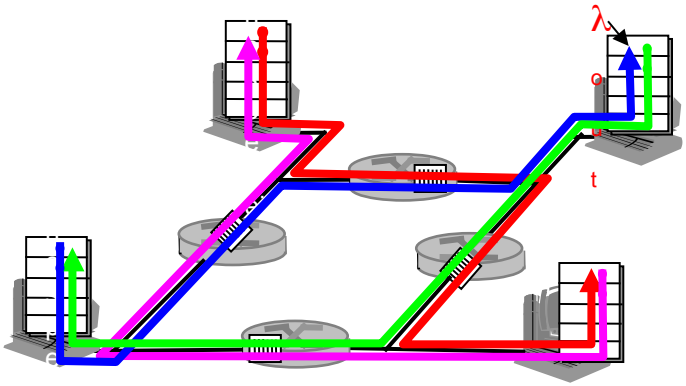
- Quatro transmissores
- Caminhos com múltiplos saltos
- Temporizações/retransmissões

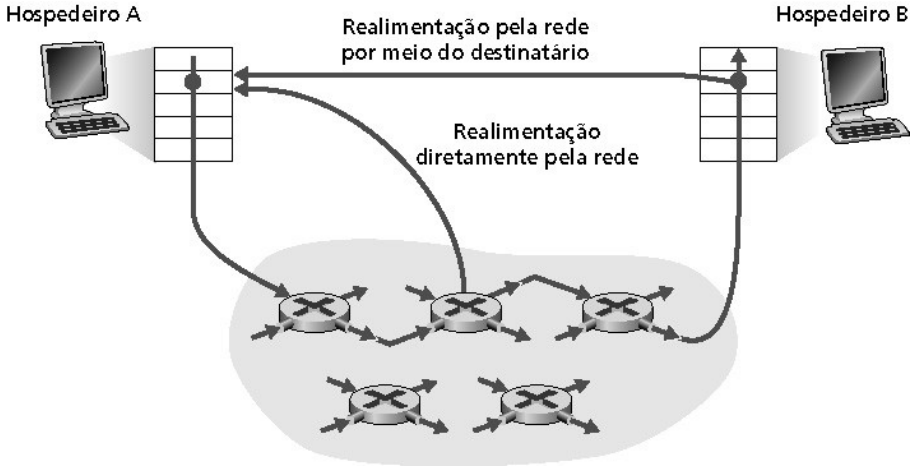
P.: o que acontece quando λ_{in} e λ'_{in} aumentam?



Causas/custos do congestionamento: cenário 3







02-068
AW/Kurose and Ross
Computer Networking
KR 03.47 ar2
15p6 Wide x 11p Deep
2/c
05/14/02GM 6/03/02GM

Outro "custo" do congestionamento:

- Quando o pacote é descartado, qualquer capacidade de transmissão que tenha sido anteriormente usada para aquele pacote é desperdiçada!

Abordagens do problema de controle de congestionamento

Existem duas abordagens gerais para o problema de controle de congestionamento:

Controle de congestionamento fim-a-fim:

- Não usa realimentação explícita da rede
- Congestionamento é inferido a partir das perdas e dos atrasos observados nos sistemas finais
- Abordagem usada pelo TCP

Controle de congestionamento assistido pela rede:

- Roteadores enviam informações para os sistemas finais
  - *Bit* único indicando o congestionamento (e.g., SNA, DECbit, ECN, ATM)
  - Taxa explícita do transmissor poderia ser enviada

Camada de transporte

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado à conexão: UDP
- 3.4 Princípios de transferência confiável de dados
- 3.5 Transporte orientado à conexão: TCP
  - Estrutura do segmento
  - Transferência confiável de dados
  - Controle de fluxo
  - Gerenciamento de conexão
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

TCP: controle de congestionamento

- Controle fim-a-fim (sem assistência da rede)
- Transmissor limita a transmissão:

  `LastByteSent-LastByteAcked ≤ CongWin`
- Aproximadamente,

$$\text{rate} = \frac{\text{CongWin}}{\text{RTT}} \quad \text{Bytes/s}$$

- **`CongWin`** é dinâmico, função de congestionamento das redes detectadas

Como o transmissor detecta o congestionamento?

- Evento de perda = ocorreu esgotamento de temporização (*timeout*)
- ou 3 ACKs duplicados

Regra geral: **Transmissor TCP reduz a taxa (`CongWin`) após o evento de perda**

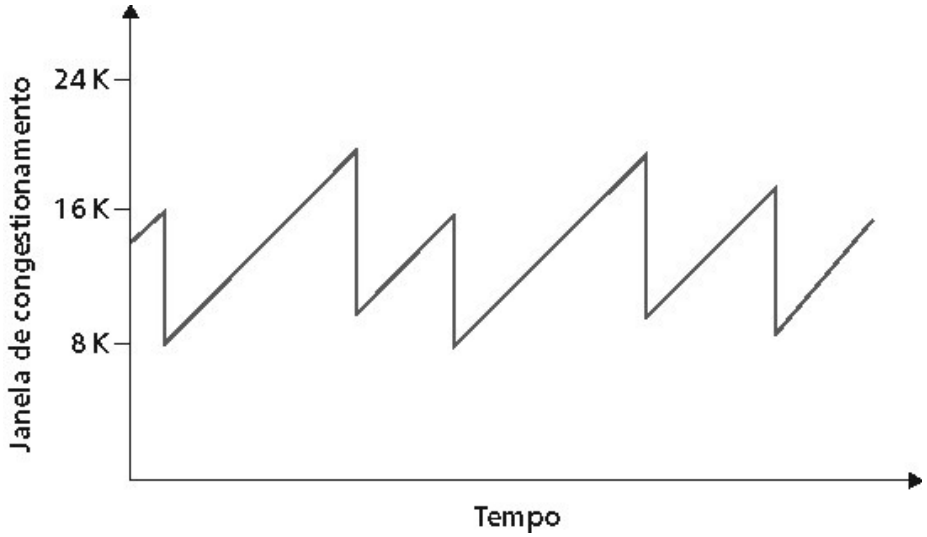
Três mecanismos:

- *Additive Increase Multiplicative Decrease* (AIMD)
- Partida lenta
- Reação a eventos de esgotamento de temporização

TCP AIMD

Redução multiplicativa: diminui o **`CongWin`** pela metade após o evento de perda

Aumento aditivo: aumenta o **`CongWin`** com 1 MSS a cada RTT na ausência de eventos de perda: ***probing***



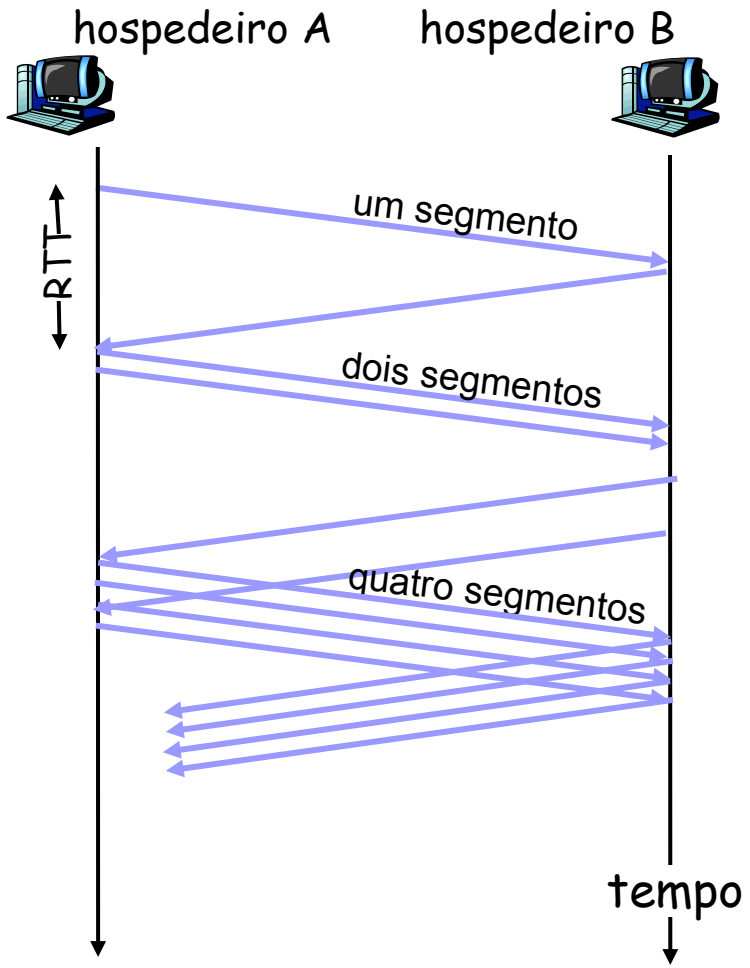
conexão TCP de longa-vida (dente-de-serra)

TCP Partida lenta

- Quando a conexão começa, **CongWin** = 1 MSS (*Maximum Segment Size*)
  - Exemplo: MSS = 500 bytes e RTT = 200 milissegundos
  - Taxa inicial = 20 kbps
- Largura de banda disponível pode ser >> MSS/RTT
  - Desejável aumentar rapidamente até a taxa respeitável
- Quando a conexão começa, a taxa aumenta rapidamente de modo exponencial até atingir um limite pré-fixado (*threshold*) OU a ocorrência do primeiro evento de perda

TCP Partida lenta

- Quando a conexão começa, a taxa aumenta rapidamente de modo exponencial até a ocorrência do primeiro evento de perda :
  - Dobra o **`CongWin`** a cada RTT
  - Faz-se incrementando o **`CongWin`** para cada ACK recebido
- Sumário: taxa inicial é lenta mas aumenta de modo exponenciamente rápido



Refinamento

- Após 3 ACKs duplicados:
  - **`CongWin`** é cortado pela metade
  - Janela então cresce linearmente
- Mas após evento de esgotamento de temporização (*timeout*):
  - **`CongWin`** é ajustado para 1 MSS;
  - A janela cresce exponencialmente até um limite (*threshold*) e após cresce linearmente

Filosofia

- 3 ACKs duplicados indica que a rede é capaz de entregar alguns segmentos
- Tempo de confirmação antes dos 3 ACKs duplicados é “mais alarmante”

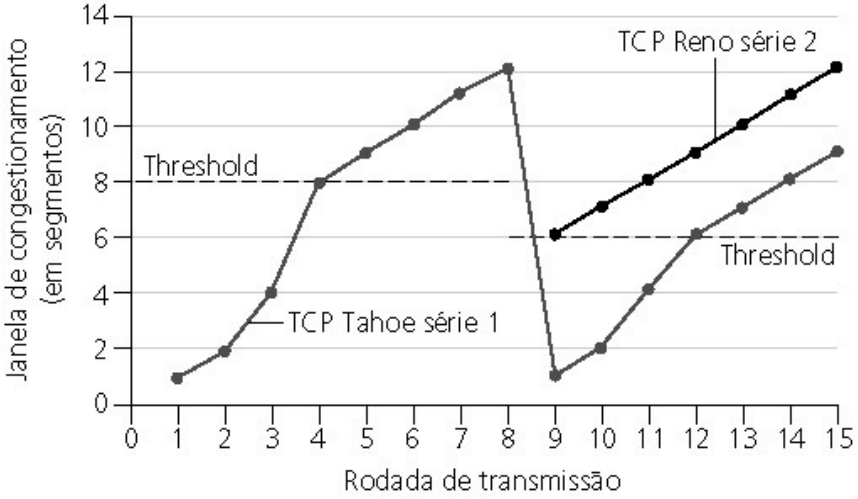
Refinamento

P.: Quando o aumento exponencial deve tornar-se linear?

R.: Quando **`CongWin`** obtiver 1/2 do seu valor antes do *timeout* de confirmação.

Implementação:

- Limite variável
- No evento de perda, o limiar é ajustado para 1/2 do CongWin logo antes do evento de perda



Resumo: controle de congestionamento TCP

- Quando **CongWin** está abaixo do limite (***Threshold***), o transmissor em fase de *slow start*, a janela cresce exponencialmente.
- Quando **CongWin** está acima do limite (**Threshold)**, o transmissor em fase de *congestion avoidance*, a janela cresce linearmente.
- Quando ocorrem três ACK duplicados, o limiar (***Threshold***) é ajustado em **CongWin/2** e **CongWin** é ajustado para ***Threshold*** (i.e., ambos assumem o mesmo valor).
- Quando ocorre tempo de confirmação, o ***Threshold*** é ajustado para **CongWin/2** e o **CongWin** é ajustado para 1 MSS.

TCP sender congestion control

Evento	Estado	Ação do transmissor	Comentário
ACK recebido para dado previament e não-confirmado	partida lenta (SS)	CongWin = CongWin + MSS, If (CongWin > Threshold) ajusta estado para "prevenção de congestionamento"	Resulta em dobrar o CongWin a cada RTT
ACK recebido para dado previament e não-confirmado	prevenção de congestiona-mento (CA)	CongWin = CongWin + MSS * (MSS/CongWin)	Aumento aditivo, resulta no aumento do CongWin em 1 MSS a cada RTT
Evento de perda detectado por três ACKs duplicados	SS or CA	Threshold = CongWin/2, CongWin = Threshold, Ajusta estado para "prevenção de congestionamento"	Recuperacao rápida, implementando redução multiplicativa o CongWin não cairá abaixo de 1 MSS.
Tempo de confirmação	SS or CA	Threshold = CongWin/2, CongWin = 1 MSS, Ajustar estado para "partida lenta"	Entra em partida lenta
ACK duplicado	SS or CA	Incrementa o contador de ACK duplicado para o segmento que está sendo confirmado	CongWin e Threshold não mudam

TCP: ***throughput***

- Qual o ***throughput*** (vazão) médio do TCP como uma função do tamanho da janela e do RTT? (ignore a partida lenta)
- Deixe W ser o tamanho da janela quando ocorre perda.
- Quando a janela é **W**, o ***throughput*** é W/RTT
- Logo após a perda, a janela cai para W/2, e o ***throughput*** para W/2RTT
- ***Throughput*** médio: 0,75 W/RTT

Futuro do TCP

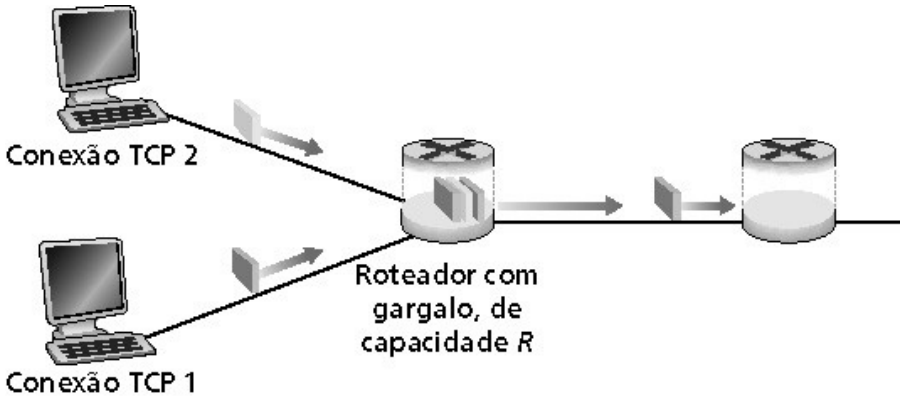
- Exemplo: segmento de 1500 bytes, RTT de 100 ms, deseja 10 Gbps de *throughput*
- Requer tamanho de janela W = 83333 para os segmentos em trânsito
- *Throughput* em termos da taxa de perda (L é a taxa de erro):

$$\frac{1.22 \cdot MSS}{RTT\sqrt{L}}$$

- → L = $2 \cdot 10^{-10}$ Uau! (ou seja, em média um erro a cada 5 bilhões de transmissões!)
- **São necessárias novas versões de TCP para alta velocidade!**

Eqüidade do TCP

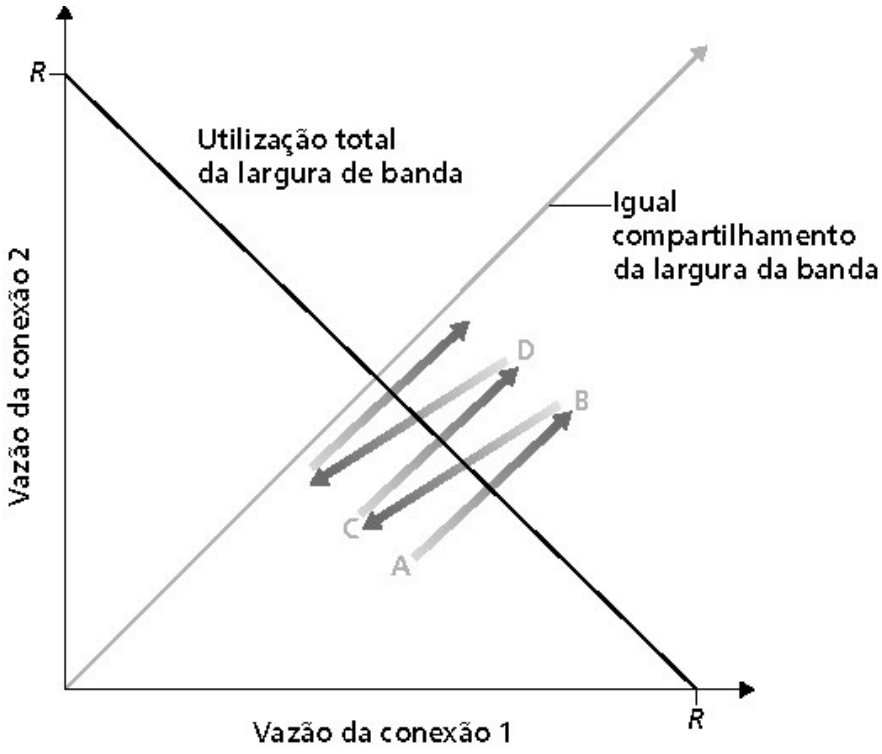
Objetivo de eqüidade: se K sessões TCP compartilham o mesmo enlace do gargalo com largura de banda R, cada uma deve ter taxa média de R/K



Por que o TCP é justo?

Duas sessões competindo pela banda:

- O aumento aditivo fornece uma inclinação de 1, quando a vazão aumenta
- Redução multiplicativa diminui a vazão proporcionalmente



perda: reduz janela por um fator de 2

prevenção de congestionamento: aumento aditivo

Eqüidade

Eqüidade e UDP

- Aplicações multimídia normalmente não usam TCP
  - Não querem a taxa estrangulada pelo controle de congestionamento
- Em vez disso, usam UDP:
  - Trafega áudio/vídeo a taxas constantes, toleram perda de pacotes

Eqüidade e conexões TCP paralelas

- Nada previne as aplicações de abrirem conexões paralelas entre 2 hospedeiros.
- *Web browsers* fazem isso
- Exemplo: enlace de taxa R atualmente com 9 conexões;
  - Uma nova aplicação pede 1 conexão TCP, obtém taxa de R/10
  - ou a aplicação pede 11 TCPs, obtém em torno de R/2

Resumo

- Princípios por trás dos serviços da camada de transporte:
  - Multiplexação/demultiplexação
  - Transferência de dados confiável
  - Controle de fluxo
  - Controle de congestionamento
- Instanciação e implementação na Internet
  - UDP
  - TCP